

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Secret Key (symmetric) Cryptography

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Cryptography

- Study of mathematical techniques related to information and communication security in the presence of third party adversaries
- The most widely used tool used by different security services
  - **not the only one**
- Can be used for:
  - **Confidentiality**
  - **Data integrity**
  - **Authentication**
  - **Non-repudiation**

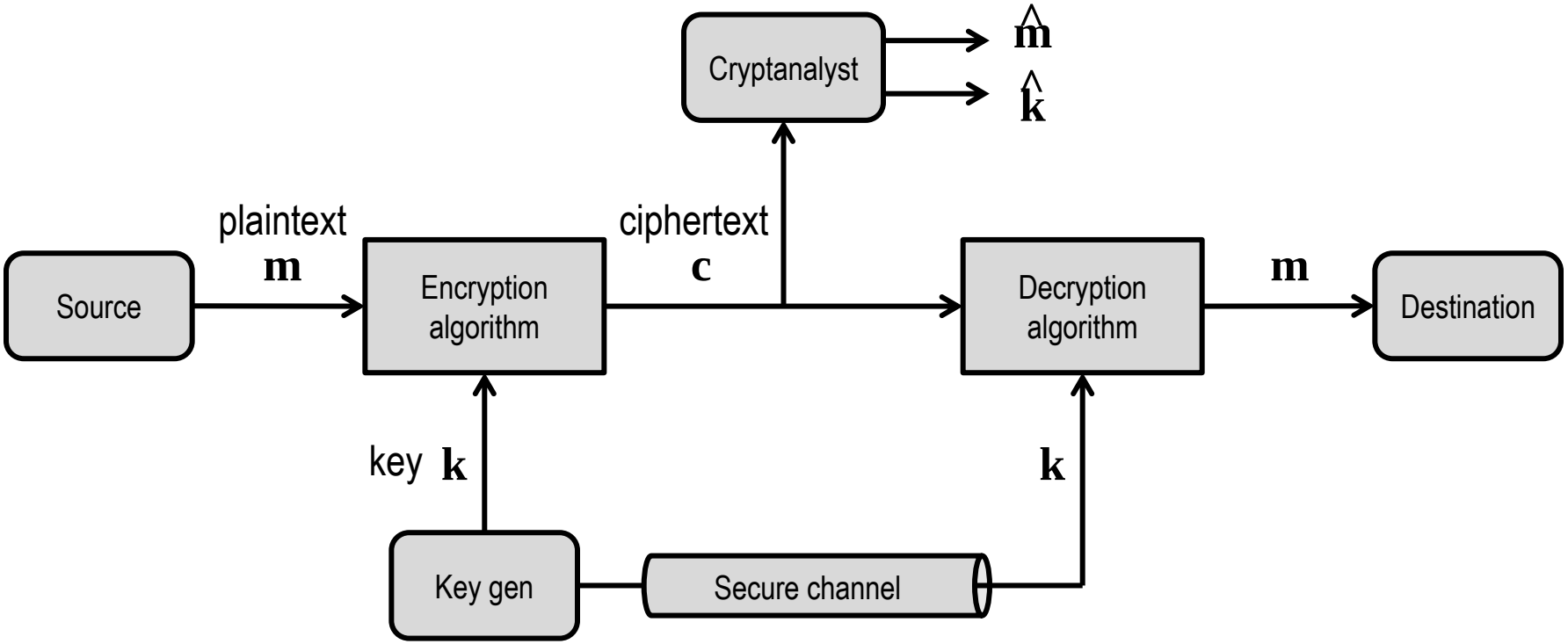
Different cryptographic algorithms

- Symmetric cryptography (Secret key cryptography)
  - **the two communication parties share a common secret (key)**
- Hash and MAC algorithms
  - **one-way transformation that maps a variable length message to a fixed length bit string**
  - **a variant is a MAC function**
    - includes a key
- Asymmetric cryptography (Public key cryptography, or private/public key cryptography)
  - **two different keys are used for two opposite functions (e.g. encryption and decryption)**
  - **one key can be publicly available (public key); the other is maintained secret for the owner (private key)**

Symmetric Cryptography

- Or conventional / secret-key / single-key
  - **sender and recipient share a common key**
- All classical encryption algorithms are secret-key
  - **was the only type prior to invention of public-key in 1970's**
- Generally used for protecting (through encryption) some data stored in a repository or sent to a remote entity
- Designed to take a reasonable-length key (e.g. 128 bits) and generating a one-to-one mapping from cleartext to ciphertext that "looks like completely random", to someone doesn't know the key

Symmetric Cipher Model



Symmetric Cipher Model

- Cipher
  - **algorithm for transforming plaintext to ciphertext**
- Two functions:
  - **Encryption (or enciphering)**
    - converting plaintext m (the original message) to ciphertext c (the encoded message)
    - uses a secret key k (known only to sender/receiver)

      c = E(k,m) = Ek(m)
      - can be either a deterministic or randomized function
  - **Decryption (or deciphering)**
    - recovering ciphertext from plaintext

      m = D(k,c) = Dk(c) = Dk(Ek(m))
      - deterministic
- Assume encryption (and decryption) algorithm is known
- Implies a secure channel (or method) to distribute key
- Examples of common algorithms are DES, 3DES, AES, ChaCha20

Symmetric Cipher Model (cont.)

- In theory, the security of a cipher might rest in the secrecy of a restricted (secret) encryption algorithm
  - $c = E(m)$, $m = D(c)$

  however:
  - whenever a user leaves a group, the algorithm must change
  - could be scrutinized by people smarter than you
- In practice, the encryption algorithm is usually not secret and uses keys
  - $c = E_k(m)$, $m = D_k(c)$
  - change of authorized participants requires only a change in key
  - the key is selected from a large set (a keyspace), e.g., a 256-bit number $\rightarrow 2^{256} \approx 10^{77}$ values!
    - robustness is at most proportional to the key length (e.g. 40 bit is weak, 128 bit strong)

➢ **Kerckhoffs' principle**
- Security should be based on secrecy of the key, not the details of the algorithm
  - Jean Guillaume Hubert Victor Francois Alexandre Auguste Kerckhoffs von Nieuwenhof, "La Criptographie Militaire", 1883

History

- In the past, a lot of (symmetric) cryptographic algorithms have been proposed and used
- Some classical (pre-computer era) examples:
  - **Substitution ciphers**
    - Caesar cipher
    - Monoalphabetic substitution
    - Vigenère cipher (actually the Bellaso cipher)
      - Bellaso cipher (1553) → repeating keyword polyalphabetic substitution
      - Vigenère cipher (1586) → autokey polyalphabetic cipher
    - Playfair cipher





  - **Electro-Mechanical ciphers**
    - Enigma (Germany, WWII)
    - Hagelin machines (Sweden, 1925-1970)

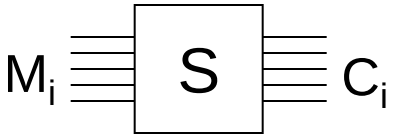




Classical encryption techniques

Substitution Ciphers

- Substitution is a classical encryption technique
- Symbols (e.g. letters) of plaintext are replaced by other symbols (letters, numbers, same, or other symbols)
- If plaintext is viewed as a sequence of bits, then substitution involves replacing plaintext bit blocks with ciphertext bit blocks



M	*C*
0000	1101
0001	1001
0010	0111
0011	1000
⋮	⋮
1111	0011

Substitution Table

- Examples of classical substitution ciphers:
  - **monoalphabetic substitution with shift (e.g. Caesar cipher)**
  - **monoalphabetic substitution (monoalphabetic cipher)**
  - **polialphabetic substitution (polialphabetic cipher)**

Caesar Cipher

- Earliest known substitution cipher
  - **by Julius Caesar**
  - **first attested use in military affairs**
  - **it is a monoalphabetic substitution with shift**
- Replaces each letter by 3rd letter on
- Example:

```
meet me after the toga party
PHHW PH DIWHU WKH WRJD SDUWB
```

Caesar Cipher (cont.)

- Can define transformation (substitution) as:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

- Mathematically give each letter a number

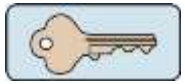
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

- Then have Caesar cipher as:

  $C = E(M) = (M + k) \bmod 26$, with k=3

  $M = D(C) = (C - k) \bmod 26$, with k=3

- If *k* is generic (and secret), we have a *Shift cipher*
  - **k is the key, with $K \in \{0,1,\ldots,25\}$**



- ROT13 is another special case of Shift cipher, with 26-letter alphabet and K=13
  - **in this case, E(X) == D(X), hence only one function ROT13(X)**

Cryptanalysis of a Shift Cipher

- Only have 26 possible ciphers
  - **A maps to A,B,..Z**
  - **If the mapping of one letter is discovered, the entire transformation is found**
- Given ciphertext, could just try all shifts of letters
  - **brute force search**
  - **e.g. break ciphertext "GCUA VQ DTGCM"**
- Do need to recognize when have plaintext

KEY	PHHW PH DIWHU WKH WRJD SDUWB
1	oggv og chvgt vjg vqic rctva
2	nffu nf bgufs uif uphb qbsuz
3	meet me after the toga party
4	ldds ld zesdq sgd snfz ozqsx
5	kccr kc ydrcp rfc rmey nyprw
6	jbbq jb xcqbo qeb qldx mxoqv
7	iaap ia wbpan pda pkcw lwnpu
8	hzzo hz vaozm ocz ojbv kvmot
9	gyyn gy uznyl nby niau julns
10	fxxm fx tymxk max mhzt itkmr
11	ewwl ew sxlwj lzw lgys hsjlq
12	dvvk dv rwkvi kyv kfxr grikp
13	cuuj cu qvjuh jxu jewq fqhjo
14	btti bt puitg iwt idvp epgin
15	assh as othsf hvs hcuo dofhm
16	zrrg zr nsgre gur gbtn cnegl
17	yqqf yq mrfqd ftq fasm bmdfk
18	xppe xp lqepc esp ezrl alcej
19	wood wo kpdob dro dyqk zkbdi
20	vnnc vn jocna cqn cxpj yjach
21	ummb um inbmz bpm bwoi xizbg
22	tlla tl hmaly aol avnh whyaf
23	skkz sk glzkx znk zumg vgxze
24	rjjy rj fkyjw ymj ytlf ufwyd
25	qiix qi ejxiv xli xske tevxc

Monoalphabetic Substitution Ciphers

- Rather than just shifting the alphabet could shuffle (permute) the letters arbitrarily
- Each plaintext letter maps to a different random ciphertext letter
- Example:
  - **Substitution table:**

abcdefghijklmnopqrstuvwxyz
DKVQFIBJWPESCXHTMYAUOLRGZN

  - **`plaintext: ifwewishtoreplaceletters`**
  - **`ciphertext: WIRFRWAJUHYFTSDVFSFUUFYA`**

- Note: the (secret) substitution can be seen as the secret key
- Now have a total of 26! ≅ 4 x 10^{26} keys
  - **with so many keys, might think is secure**
    - but would be wrong!
    - cryptanalysis based on text frequency and correlation

Cryptanalysis of Monoalphabetic Cipher

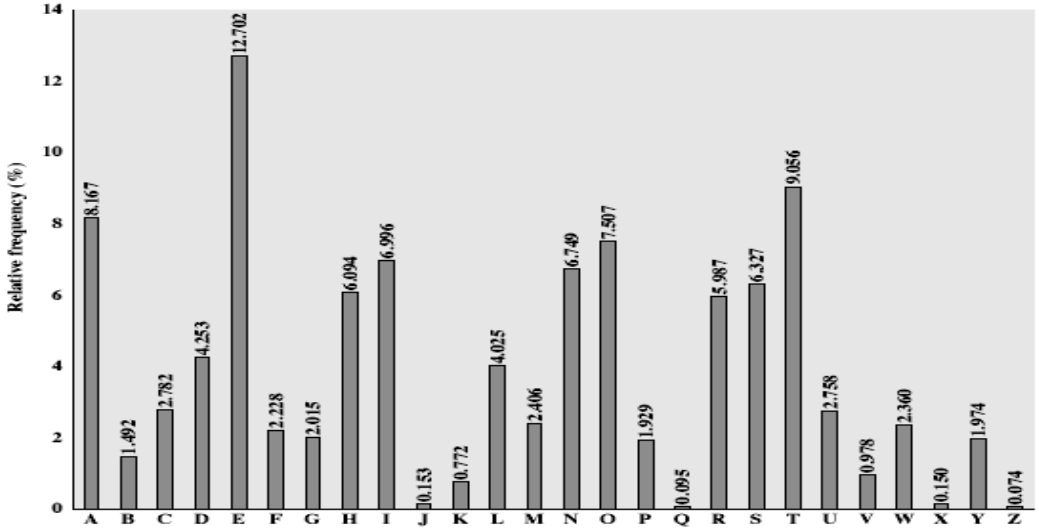
- Main problem with monoalphabetic substitutions is text redundancy
  - **non uniform frequency distributions and correlation**
- In case of human languages:
  - **letter frequencies**
    - in English 'e' is by far the most common letter, then T,R,N,I,O,A,S; other letters are fairly rare (e.g. Z,J,K,Q,X)
  - **two letters frequencies (e.g. “th” in english)**
  - **most common words**
  - **etc.**
- Cryptanalysis:
  - **calculate letter frequencies for ciphertext**
  - **compare counts/plots against known values**
  - **have tables of single, double & triple letter frequencies**
  - **discovered by Arabian scientists in 9th century**

Cryptanalysis Example

- given ciphertext:

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
```

- count relative letter frequencies



- guess *P* & *Z* are *e* and *t*
- guess *ZW* is *th* and hence *ZWP* is *the*
- proceeding with trial and error finally get:

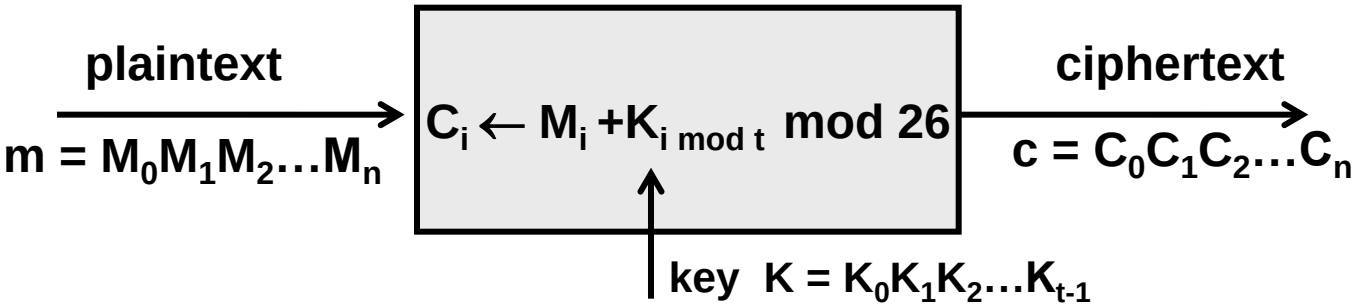
```
it was disclosed yesterday that several informal but
direct contacts have been made with political
representatives of the viet cong in moscow
```

Polyalphabetic Substitution Ciphers

- An approach to improve security is to use multiple cipher alphabets
  - **polyalphabetic substitution ciphers**
  - **uses a set monoalphabetic substitutions**
  - **defines a rule to determine which cipher alphabet (substitution) should be used at each step**
    - normally uses a key to select which substitution is used for each letter of the message
      - repeat from start after end of key is reached

Vigenère Cipher

- **Simplest polyalphabetic substitution cipher is the Vigenère Cipher (Blaise de Vigenère, 1523-1596)**



- Key is multiple letters long
  - i^{th} letter specifies i^{th} alphabet to use
  - use each alphabet in turn
- Repeat from start after *t* letters
- Effectively multiple Caesar ciphers
  - **$K = K_0 K_1 \ldots K_{t-1}$**
- Decryption works in reverse

	0	1	2	3	4	5	6	7	8	9	.	.	.	25
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
+ 1	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
+ 2	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
+ 3	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
+ 4	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
+ 5	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
+ 6	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
+ 7	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
+ 8	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
+ 9	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
+10	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
+11	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
+12	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
+13	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
+14	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
+15	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
+16	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
+17	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
+18	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
+19	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
+20	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
+21	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
+22	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
+23	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
+24	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
+25	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Vigenère Cipher (cont.)

```
(00) A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
(01) B C D E F G H I J K L M N O P Q R S T U V W X Y Z A
(02) C D E F G H I J K L M N O P Q R S T U V W X Y Z A B
(03) D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
(04) E F G H I J K L M N O P Q R S T U V W X Y Z A B C D
(05) F G H I J K L M N O P Q R S T U V W X Y Z A B C D E
(06) G H I J K L M N O P Q R S T U V W X Y Z A B C D E F
(07) H I J K L M N O P Q R S T U V W X Y Z A B C D E F G
(08) I J K L M N O P Q R S T U V W X Y Z A B C D E F G H
(09) J K L M N O P Q R S T U V W X Y Z A B C D E F G H I
(10) K L M N O P Q R S T U V W X Y Z A B C D E F G H I J
(11) L M N O P Q R S T U V W X Y Z A B C D E F G H I J K
(12) M N O P Q R S T U V W X Y Z A B C D E F G H I J K L
(13) N O P Q R S T U V W X Y Z A B C D E F G H I J K L M
(14) O P Q R S T U V W X Y Z A B C D E F G H I J K L M N
(15) P Q R S T U V W X Y Z A B C D E F G H I J K L M N O
(16) Q R S T U V W X Y Z A B C D E F G H I J K L M N O P
(17) R S T U V W X Y Z A B C D E F G H I J K L M N O P Q
(18) S T U V W X Y Z A B C D E F G H I J K L M N O P Q R
(19) T U V W X Y Z A B C D E F G H I J K L M N O P Q R S
(20) U V W X Y Z A B C D E F G H I J K L M N O P Q R S T
(21) V W X Y Z A B C D E F G H I J K L M N O P Q R S T U
(22) W X Y Z A B C D E F G H I J K L M N O P Q R S T U V
(23) X Y Z A B C D E F G H I J K L M N O P Q R S T U V W
(24) Y Z A B C D E F G H I J K L M N O P Q R S T U V W X
(25) Z A B C D E F G H I J K L M N O P Q R S T U V W X Y
```

- Example:

  key: REBUS (17,4,1,21,18)

```
plaintext:      codic emolt osicu ro
key:            REBUS REBUS REBUS RE
ciphertext:     TSECU VQPFL FWJWM IS
```

Cryptanalysis of Vigenère Ciphers

- Polyalphabetic substitution ciphers make cryptanalysis harder
  - **have multiple ciphertext letters for each plaintext letter**
    - hence letter frequencies are obscured
      - more cipher alphabets to guess and flatter frequency distribution
- But not totally lost
  - **one weakness is the repeating nature of the key**
    - need to determine number of alphabets (key length)
    - start with letter frequencies
    - then can attack each
      - they are like shift ciphers and can be broken individually

Transposition Ciphers

- Transposition is another classical encryption technique
  - **hides the message by rearranging the letter order (blocks of bits)**
  - **without altering the actual letters used**
  - **performs a sort of permutation**
- Can recognize these since have the same frequency distribution as the original text
- Example
  - **permutation**

Example: Row Transposition Ciphers

- Write letters of message out in rows over a specified number of columns
- then reorder the columns according to some key before reading off the rows

```
Key:        4 3 1 2 5 6 7
Plaintext: a t t a c k p
           o s t p o n e
           d u n t i l t
           w o a m x y z
Ciphertext: TTNAAPTMTSUOAODWCOIXKNLYPETZ
```

Product Ciphers

- Ciphers using substitutions or transpositions may be not sufficiently secure
- Hence consider using several ciphers in succession to make harder:
  - **two substitutions make a more complex substitution**
  - **two transpositions make a more complex transposition**
  - **a substitution followed by a transposition make a new much harder cipher**
- This is bridge from classical to modern ciphers

Rotor Machines



- Before modern ciphers, rotor machines were most common product cipher
- Were widely used in WW2
  - **German Enigma, Allied Hagelin, Japanese Purple**
- Enigma uses a series of cylinders, each giving one substitution, which rotated and changed after each letter was encrypted
  - **every key press caused one or more rotors to step by one**
  - **implements a varying substitution cipher**
    - polyalphabetic substitution cipher
- With 3 cylinders have 26x26x26=26^3=17576 alphabets

Some online resources

- Crypto Museum
  - **https://www.cryptomuseum.com/crypto/**

- Online ciphers
  - **https://www.dcode.fr**
    - e.g. https://www.dcode.fr/vigenere-cipher
  - **https://cryptii.com/pipes**
    - e.g. https://cryptii.com/pipes/vigenere-cipher

Exercises

- Crypoanalysis of Vigenère cipher
  - **Encrypt a message using Vigenère's cipher. Write a program or use an online tool like**
    - https://cryptii.com
  - **Try to decrypt the resulting ciphertext knowing only the key length. For this purpose you could try to use**
    - https://www.dcode.fr/vigenere-cipher
- Example:
  - **Ciphertext: Llkj ml s zgic lwgtvx fwwurkx. Vsp'k xxdp cechfi.**
  - **Alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ**
  - **Key length: 6**

Perfect Secrecy

Communication Theory of Secrecy Systems



- Claude Elwood Shannon (1916-2001)
- "Communication Theory of Secrecy Systems"
  - **Bell System Technical Journal, vol.28(4), page 656–715, 1949**
    - http://netlab.cs.ucla.edu/wiki/files/shannon1949.pdf
    - can be found also at:
      - https://web.archive.org/web/20201112040412/http://netlab.cs.ucla.edu/wiki/files/shannon1949.pdf
      - https://pages.cs.wisc.edu/~rist/642-spring-2014/shannon-secrecy.pdf
- First mathematical theory of cryptography
  - **security as information leakage problem**
  - **birth of modern cryptography**
- It follows the work "A mathematical theory of communication", published in 1948
    - fundamental work on information and communication theory

Shannon's model of a cryptosystem

- A secrecy system consists of:
  - **a message space** M
  - **a key space** K
  - **a ciphertext space** C
  - **two functions**
    - Enc: $\mathrm{K} \times \mathrm{M} \rightarrow \mathrm{C}$
    - Dec: $\mathrm{K} \times \mathrm{C} \rightarrow \mathrm{M}$
- The two functions define two sets of transformations E_k and D_k indexed by keys $k \in K$
  - **encryption transformations** $\mathrm{E_k}: \mathrm{M} \rightarrow \mathrm{C}$
  - **decryption transformations** $\mathrm{D_k}: \mathrm{C} \rightarrow \mathrm{M}$
- Correctness condition:
  - $\mathrm{D_k}(\mathrm{E_k}(\mathrm{m})) = \mathrm{m}$
- Two secrecy systems will be the same if they consist of the same set of transformations E_k

Theoretical vs practical secrecy

- Shannon distinguishes between:
  - **Practical secrecy**
    - breaking the system requires infeasible effort
  - **Theoretical secrecy**
    - impossible to break the system even with unlimited time and manpower available for the analysis of intercepted cryptograms
    - the ciphertext provides no information about the plaintext

Probabilistic Modeling

- Shannon's key conceptual move:
  - **Treat messages as outcomes of a random variable** (M)
  - **Treat keys as outcomes of a random variable** (K)
  - **Assume** (M) **and** (K) **are independent**
- Ciphertext is $C = E_K(M)$
- Thus the secrecy system becomes a probabilistic transformation:

$$(\mathrm{M},\mathrm{K}) \rightarrow \mathrm{C}$$

  - **this transforms cryptography into an information-theory problem**
- Then a secrecy system can be described as conditional probability distributions
  - **this is the most useful representation for theoretical secrecy**

a-priori and *a-posteriori* probabilities

- A Priori Probability
  - **before observing ciphertext**

$$\Pr(M=m)$$

  - **this reflects data statistics, redundancy, source structure**
- A Posteriori Probability
  - **after observing ciphertext** $C=c$

$$\Pr(M=m \mid C=c)$$

  - **this is the attacker's updated belief**

- Secrecy is about how much this posterior differs from the prior $Pr(M=m)$

Perfect Secrecy Definition

- A system has **perfect secrecy**, if observing ciphertext should have *no effect* on the adversary's knowledge regarding the actual message that was sent
  - **the *a posteriori* probability that some message *m* $\in$ *M* was sent, conditioned on the ciphertext that was observed, should be no different from the *a priori* probability that *m* would be sent**
- That is, a system has **perfect secrecy** if for all (m, c) such that $Pr(C=c)>0$,

$$\Pr(\mathrm{M}=\mathrm{m}|\mathrm{C}=\mathrm{c}) = \Pr(\mathrm{M}=\mathrm{m})$$

  - **ciphertext does not change the probability distribution of messages**
  - **equivalently, M and C are independent**

Perfect Secrecy Theorem

- A secrecy system has perfect secrecy if and only if

$$\Pr(C=c \mid M=m) = \Pr(C=c) \;, \forall\; m,c$$

  - **obtained using Bayes' theorem**

$$\Pr(M=m \mid C=c) = \Pr(C=c \mid M=m)\, \Pr(M=m) \,/\, \Pr(C=c)$$

    and perfect secrecy definition

$$\Pr(M=m \mid C=c) = \Pr(M=m)$$

- That is $Pr(C=c \mid M=m)$ must be independent of m
  - ciphertext distribution is independent of plaintext
  - **each message must produce ciphertexts according to the same distribution**

Perfect secrecy implies every ciphertext is compatible with every message

- Take a ciphertext value c such that $Pr[C=c]>0$
- Take any message $m \in M$. Since $Pr[M=m]>0$, perfect secrecy says $Pr[M=m \mid C=c] = Pr[M=m]$, then $Pr[M=m \mid C=c] >0$, and:

$$\Pr[\mathrm{M}=\mathrm{m}\ ,\ \mathrm{C}=\mathrm{c}] > 0$$

- But the event $C=c$ happens only when there exists some key k such that $E(k,m)=c$. Therefore:
- For this c, **for every** message m, there exists **at least one** key k with

$$\mathrm{E}(\mathrm{k},\mathrm{m})=\mathrm{c}$$

- So each "good" ciphertext c has at least one key mapping each plaintext to it. This implies that

$$|\mathrm{K}| \geq |\mathrm{M}|$$

Introduction of Entropy

- Shannon imports concepts from information theory
- For a random variable X, The Entropy $H(X)$ is:

$$\mathrm{H}(\mathrm{X}) = -\sum_{\mathrm{x}} \Pr(\mathrm{x}) \log \Pr(\mathrm{x})$$

- Interpretation:
  - **measure of uncertainty**
  - **average information content**

- In case of M,
  - $\mathrm{H}(\mathrm{M})$ = a priori uncertainty of the message

Conditional Entropy

- Conditional Entropy (also called as Equivocation):

$$H(M|C) = -\sum_{m,c} \Pr[M=m,C=c] \log \Pr[M=m|C=c]$$

  - **that is also equal to**

$$= -\sum_{c} \Pr[C=c] \sum_{m} \Pr[M=m|C=c] \log \Pr[M=m|C=c] =$$

$$= \sum_{c} \Pr[C=c] \, H(M \mid C=c)$$

- It measures the average remaining uncertainty about message after observing ciphertext
  - **a posteriori uncertainty of the message**

Perfect Secrecy

- A cryptosystem has perfect secrecy if and only if

$$H(M) = H(M|C)$$

  - **Proof**
    - Conditional entropy definition

$$H(M|C)=\sum_c \Pr[C=c]\, H(M \mid C=c)$$

    - where

$$H(M|C=c)=-\sum_m \Pr[M=m \mid C=c] \log \Pr[M=m \mid C=c]$$

    - If $\Pr[C=c]>0$, then $\Pr[M=m \mid C=c] = \Pr[M=m]$
    - So for every such c,

$$H(M \mid C=c) = -\sum_m \Pr[M=m] \log \Pr[M=m] = H(M)$$

    - Therefore,

$$H(M|C) = \sum_c \Pr[C=c]\, H(M \mid C=c) =$$
$$= \sum_c \Pr[C=c]\, H(M) = H(M) \sum_c \Pr[C=c] =$$
$$= H(M)$$

Upper Bound on Secrecy

- One of the most important results, an upper bound on secrecy:

$$H(M|C) \leq H(K)$$

- Meaning:
  - **the uncertainty remaining about the message after observing ciphertext cannot exceed the entropy of the key**
  - **key entropy limits achievable secrecy**
- Perfect Secrecy Implies that, if

$$H(M|C) = H(M)$$

  - **then**

$$H(K) \geq H(M)$$

  - **this is the famous Shannon bound**

Interpretation of the Shannon upper bound

- To conceal all information about the message:
  - **the key must carry at least as much uncertainty as the message**
- Thus:
  - **perfect secrecy requires key entropy ≥ message entropy**
  - **short keys cannot provide perfect secrecy**
- This result is purely information-theoretic
  - **no computational assumptions**

Threat models and attacks

Threat model

- In general an adversary may try to:
  - **deduce the key used from a specific plain text, to compromise all future and past messages encrypted with that key**
  - **to guess the plain text from the encrypted text**
- Leveraging on:
  - **the knowledge of the encryption algorithm**
  - **some knowledge of the general characteristics of plaintext**
  - **(possibly) some sample pairs of {plaintext, ciphertext}**
- The threat model specifies what “power” the attacker is assumed to have, without placing any restrictions on the adversary’s strategy
- Plausible options for the threat model are:
  - **Know-ciphertext (Ciphertext-only) attack (KCA)**
  - **Known-plaintext attack (KPA)**
  - **Chosen-plaintext attack (CPA)**
  - **Chosen-ciphertext attack (CCA)**

Ciphertext-only Attack (KCA)

- This is the most basic attack, where the adversary just observes (and presumably stores) some ciphertext (or multiple ciphertexts) that can be analyzed
  - **he/she attempts to determine information about the underlying plaintext and/or key**
  - **it is necessary to have enough ciphertext**

- It is the hardest attack to carry on
  - **the opponent has the least amount of information to work with**

- Sometimes the adversary might be able to recognize when he/she has successfully decrypted a ciphertext
  - **This case is often called *recognizable plaintext* attack**
    - for example in case of normal text or known document formats

Known plaintext Attack (KPA)

- The adversary is able to learn one or more plaintext/ciphertext pairs generated using some key
  - **from that pairs, the adversary can try to figure out the mapping of some fraction of the text**
- The aim is then to deduce information about the underlying plaintext of some *other* ciphertext produced using the same key
- How it is possible to obtain the plaintext?
  - **the secret data does not remain secret forever**
  - **or the opponent may have knowledge of what is in the message**
    - certain key words (e.g. in the header of the message)
      - expected patterns (e.g. PostScript, etc.)
    - probable-word attack
- Some cryptographic schemes might be good enough to be secure against *ciphertext only* attacks but not against to *known plaintext* attacks
  - **in these cases, it is important to minimize the possibility for a bad guy to obtain <m,c> pairs**

Chosen Plaintext Attack (CPA)

- The opponent can choose any plaintext and get the corresponding ciphertext from the system
  - **e.g. there is a transmission service that encrypts and transmits messages; the bad guy can ask the transmission service to transmit any plaintext he/she wants**

Chosen Ciphertext Attack (CCA)

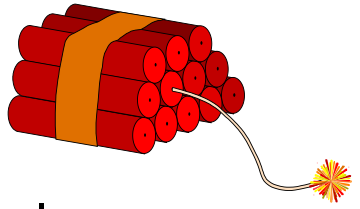
- The adversary chooses any ciphertext and is able to get the corresponding plaintext from the system

- Some cryptographic schemes might be good enough to be secure against KCAs and KPAs but not against to CPAs or CCAs

Cryptography Attacks

- There are some general approaches to attacking a conventional encryption scheme:
  - **Brute-force (search) attack**
    - tries every possible encryption/decryption (e.g. by trying all possible keys)
      - it may require the visit of all key space
      - The average number of required attempts is the half of the number of possible keys
    - it requires to be able to recognize when the correct plaintext/ciphertext has been obtained
  - **Cryptographic analysis (cryptoanalysis)**
    - based on the type of the cryptographic algorithm, tries to exploit some characteristic of the algorithm and/or properties of some previous plaintext/ciphertext pairs to deduce a plaintext and/or key
      - does not require to obtain the encryption key for deduce the plaintext
  - **Side-channel attacks**
    - use information from the physical implementation of a cryptosystem

Brute force attack



- Method of defeating a cryptographic scheme by trying a large number of possibilities
  - **exhaustive search on a large set of (or all) possibilities**
    - testing all possible keys in order to recover the plaintext used to produce a particular ciphertext (or vice-versa)
  - **most basic attack, proportional to key size**
  - **assume either know/recognise plaintext**
    - the attacker has to determine when he succeeded
      - by obfuscating the data to be encoded, brute force attacks are made less effective as it is more difficult to determine when one has succeeded in breaking the code
- In most schemes, the theoretical possibility of a brute force attack is recognized, but it is set up in such a way that it would be computationally infeasible to carry out

Brute Force Search - Example

Key size (bits)	*Number of alternative keys*	*Time required at 1 decryption/μs*	*Time required at 10^6 decryption/μs*
32	$2^{32} = 4.3 \times 10^9$	$2^{32}\mu s = 71.6$ minutes	4.3 ms
64	$2^{64} = 1.8 \times 10^{19}$	$2^{64}\mu s = 584542$ years	213 days
128	$2^{128} = 3.4 \times 10^{38}$	$2^{128}\mu s = 1.1 \times 10^{25}$ years	1.1×10^{19} years
256	$2^{256} = 1.2 \times 10^{77}$	$2^{256}\mu s = 3.7 \times 10^{63}$ years	3.7×10^{57} years

Cryptoanalysis

- Based on the type of the cryptographic algorithm, tries to exploit some characteristic of the algorithm and/or properties of some previous plaintext/ciphertext pairs
  - **to deduce a plaintext and/or key**
  - **for deducing the plaintext, may not require to obtain the key**
- Used also to to test the robustness of the algorithm and of the key
- Some techniques:
  - **Differential Cryptanalysis**
    - analyzes how differences in plaintext affect ciphertext differences
    - very effective against block ciphers
  - **Linear Cryptanalysis**
    - uses linear approximations to describe cipher behavior
    - statistical method

Side channel attack

- Any attack based on information gained from the physical implementation of a cryptosystem, rather than theoretical weaknesses in the algorithms
  - **e.g. timing information, power consumption**
- General classes of side channel attack include:
  - **Timing attack — attacks based on measuring how much time various computations take to perform**
  - **Power monitoring attack — attacks which make use of varying power consumption by the hardware during computation**
  - **TEMPEST (aka Van Eck or radiation monitoring) attack — attacks based on leaked electromagnetic radiation which can directly provide plaintexts and other information**

Side channel attack (cont.)

- In all cases, that physical effects caused by the operation of a cryptosystem can provide useful extra information about secrets in the system
  - **about the cryptographic key, partial state information, full or partial plaintexts and so forth**
- Side-channel attacks require considerable technical knowledge of the internal operation of the system on which the cryptography is implemented

Computational and Unconditional Security

- Unconditional security
  - **an encryption scheme is unconditionally secure if no matter how much computer power is available, the cipher cannot be broken**
    - Shannon definition of Perfect secrecy
    - e.g. the OTP (One Time Pad) chiper
- However cryptographic algorithms are often not impossible to attack
  - **e.g. brute force attack**
- Computational security
  - **an encryption scheme is computationally secure if given limited computing resources the cipher cannot be broken**
    - e.g. the time required to break the cipher exceeds the useful lifetime of the information
    - depends on the attack complexity and cost
      - processing complexity: a large number of operations required (long time)
      - data complexity: a large number of expected inputs (e.g., ciphertext)
      - storage complexity: a large amount of storage units required
    - requires the estimation of the attacker resources

One-Time Pad (OTP)

One-Time Pad

- One-Time Pad (OTP) cipher
  - patented by Gilbert Vernam, Bell Telephone Laboratories, in 1919
  - first described by Frank Miller (a California banker) in 1882
- OTP cipher on binary data:
  - consider a plaintext message m as sequence of bits
  - for each message m to be encrypted we need a key k (sequence of bits) as long as the message
  - the message m is encrypted combining each bit M_i of m with each bit K_i of the key k by doing the following operation:
    - $\mathrm{C_i} = \mathrm{M_i}\ \mathrm{XOR}\ \mathrm{K_i}$
      - where:

        0 xor 0 = 0
        1 xor 0 = 1
        0 xor 1 = 1
        1 xor 1 = 0
  - Similarly, M_i can be obtained back from C_i (decryption) as:
    - $\mathrm{M_i} = \mathrm{C_i}\ \mathrm{XOR}\ \mathrm{K_i}$

One-Time Pad (cont.)

- Applied to the entire message m with an OTP key k, it is:
  - $c = m \text{ XOR } k$
  - $m = c \text{ XOR } k$

- Example:

```
m= 01001000011001010110110001101100011011110010110000100000011101000 ...
k= 00010101100101010011010111010110001011101101100101001001011000001 ...
c= 01011101111100000101100110111010010000011111010101101001000101010 ...
```

in hexadecimal:

```
m= 48656c6c6f2c207468697320697320616e206578616d706c65
k= 159535d62ed94961c45b5019d2717f0ab74c614549e3c1911d
c= 5df059ba41f56915ac322339bb025f6bd96c043d288eb1fd78
```

One-Time Pad (cont.)

- If the key is
  - **as long as the message being encrypted,**
  - **completely random, meaning every possible key is equally likely,**
  - **used only once (hence "one-time" pad),**
- the OTP provides the so-called perfect secrecy, that is it is unbreakable
  - **mathematically proved by Claude Shannon in 1945 (classified, published later in 1949)**

OTP perfect secrecy

- A cipher has **perfect secrecy** if seeing the ciphertext gives you *no information whatsoever* about the plaintext
  - **formally:**

$$\Pr(\mathrm{M}|\mathrm{C})=\Pr(\mathrm{M})$$

  - **this means:**
    - the probability a message was sent is the same, even after seeing the ciphertext

OTP perfect secrecy

- Assume:
  - **messages are n bits long**
  - **key is random n-bit string (total number of possible keys: $N=2^n$)**
- Suppose the attacker sees a ciphertext c
  - **for each possible message m_i there is exactly one key k_i such that $c=m_i \oplus k_i$, that is: $k_i = c \oplus m_i$**
  - **if keys are uniform ($Pr(K=k)=1/N$), it is $Pr(C=c \mid M=m)=1/N$**
  - **compute $Pr(C=c)$**

$$\Pr(\mathrm{C}=\mathrm{c}) = \sum_{\mathrm{m}} \Pr(\mathrm{C}=\mathrm{c} \mid \mathrm{M}=\mathrm{m}) \Pr(\mathrm{M}=\mathrm{m}) = \sum_{\mathrm{m}} (1/\mathrm{N}) \Pr(\mathrm{M}=\mathrm{m}) = \\ = (1/\mathrm{N}) \sum_{\mathrm{m}} \Pr(\mathrm{M}=\mathrm{m}) = 1/\mathrm{N}$$

  - **hence**

$$\Pr(\mathrm{C}=\mathrm{c} \mid \mathrm{M}=\mathrm{m}) = \Pr(\mathrm{C}=\mathrm{c})$$

  - **that implies perfect secrecy**
    - from Shannon's Perfect Secrecy Theorem
      - obatined by applying Bayes's Theorem

One-Time Pad (cont.)

- OTP can be seen as a special case of Vigenère cipher:
  - plaintext messages as sequence of symbols from a given alphabet $\mathcal{A}$ of size *t*
  - for simplicity consider keys formed as sequences of symbols of the same alphabet $\mathcal{A}$, or equivalently, as sequence of numbers with values from *0* to *t-1*
  - for each message *m* to be encrypted we need a key *k* as long as the message
    - $k=\{K_0,K_1,K_2,..,K_{t-1}\}$
  - the message *m* is encrypted combining each symbol M_i of the message with each symbol/number K_i of the key by doing the following addition (shifting) modulo *t*:

$$C_i = M_i + K_i \bmod t$$

  - decryption is performed by doing the reverse operation, that is:

$$M_i = C_i - K_i \bmod t$$

One-Time Pad (cont.)

- In case of binary data, the alphabet is $\mathcal{A}$ ={0,1}
- In this case the encryption is
  - $C_i = M_i + K_i \bmod 2$
- that leads only to two possible substitutions:
  - {0,1}→{0,1}, with K_i=0
  - {0,1}→{1,0}, with K_i=1
- That is equivalent to:
  - $C_i = M_i$ XOR K_i

Limitations

- While OTP is perfectly secure, it has practical limitations in terms of Key Management
  - **the key must be as long as the sum of all messages, so for long communications, the key management becomes infeasible**
    - Key Distribution
      - the key must be generated and securely distributed/exchanged between the parties
    - Storage
      - both sender and receiver must securely store the key,

Cryptographic primitives

- Most of the cryptographic systems and algorithms are designed based on some basic cryptographic functions/primitives like:
  - **Keyed functions**

    $F_k : \{0,1\}^n \rightarrow \{0,1\}^m$
    - stateless and deterministic
    - not necessarily invertible
    - output length (usually) fixed
  - **Keyed permutations**

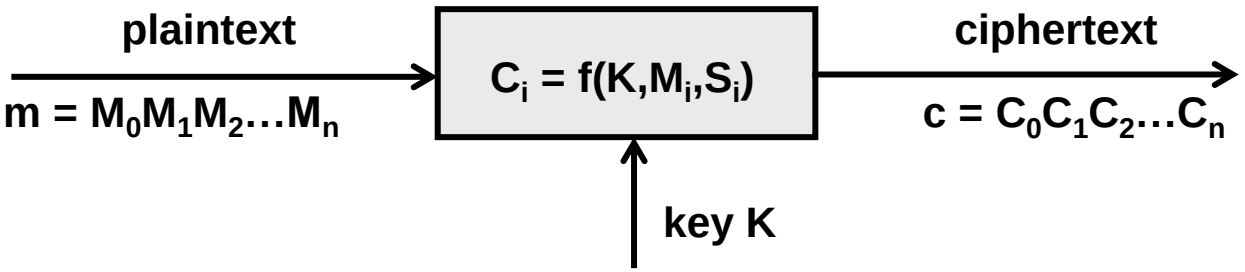
    $P_k : \{0,1\}^n \rightarrow \{0,1\}^n$
    - bijective substitution
  - **Unkeyed compression functions**
    - $C : \{0,1\}^a \rightarrow \{0,1\}^b$ (with a>b)
  - **Stream generators**

    $G : \{0,1\}^* \rightarrow \{0,1\}^*$

Stream and Block Ciphers

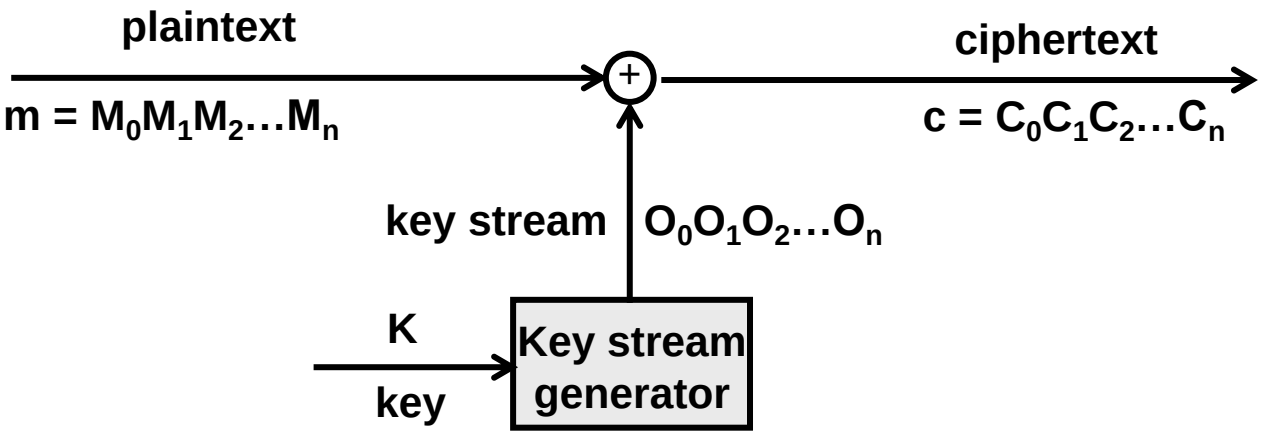
Stream ciphers

- In general we want to encrypt variable length messages
  - **arbitrary amount of information (any size)**
- We need a practical way for processing possibly long messages
- Encrypt the input message *m* by processing it step by step
  - **the message m is seen as a (possibly long) sequence of blocks M_i**
    - a block could be a bunch of bytes, a single byte, or a bit
  - **accordingly the output is a sequence of blocks C_i**
  - **C_i may be function of the current input M_i, an internal state S_i, and the secret key K**
  - **the internal state S_i may be a function of previous M_j (or C_j), with $j<i$**



Stream ciphers (cont.)

- A very common scheme for stream ciphers is:



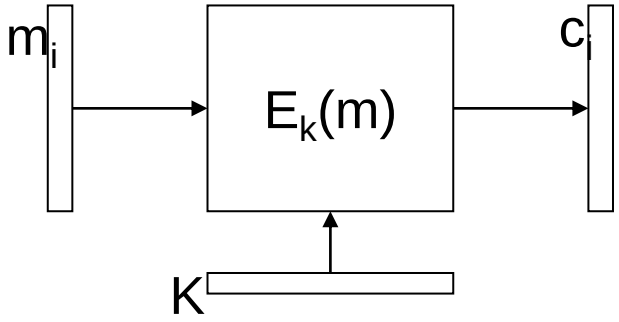
- Need a keystream as long as the message (OTP)
- Example, ChaCha20 stream cipher
- Most stream ciphers are based on cryptographic-secure pseudorandom number generators (CS-PRNG)
  - **the key is used to initialize the generator, and either key bytes or plaintext bytes are fed back into the generator to produce the byte stream**

PRNG and CSPRNG

- PRNG (Pseudo random number generator)
  - **generates a sequence of random bits (or numbers), starting from an initial value, called the seed**
  - **the sequence appears to be like randomly generated with a uniform distribution**
  - **however the sequence is deterministic, based on the seed**
  - **if used as the key stream generator, a key is used to initialize the generator**
- With standard PRNG, after a surprisingly small number of observations each and every subsequent value can be predicted
  - **they are considered cryptographically insecure**
- Cryptographically Secure PRNGs (CSPRNGs) can be used
  - **next-bit guessing resistant**
    - if someone knows all k bits from the start of the PRNG, he/she would be unable to predict the bit k+1 using reasonable computing resources
  - **compromised-state previous bits resistant**
    - if the internal state it is revealed somehow, an attacker should not be able to reconstruct previous random numbers

Block ciphers

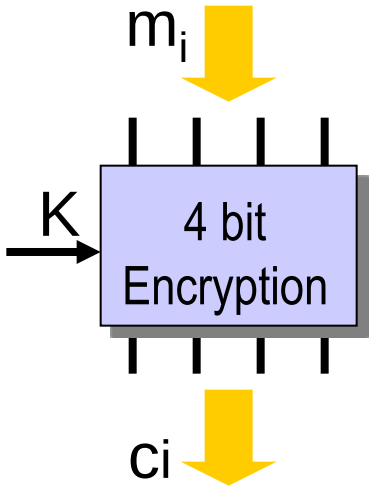
- Are very common elements for current cryptosystems
- They process messages of fixed size
  - **plaintext and ciphertext are treated as a sequence of n-bit blocks of data**
  - **ciphertext is same length as plaintext**
  - **ciphertext depends on plaintext and a key value**



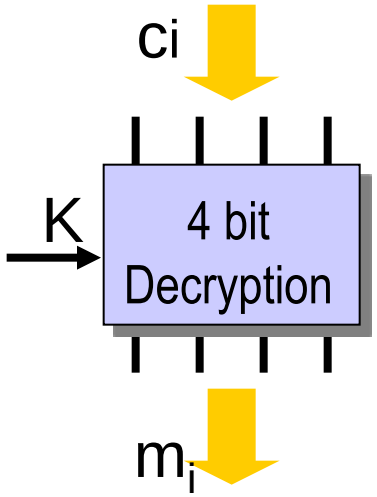
- Based on a block ciphers, there are different techniques mode for building a stream cipher
  - **e.g. CBC, OFB, CFB, and CTR modes**

Block ciphers (cont.)

- Same input blocks encrypted with the same key are transformed into the same output block
- Example: block size = 4 bits



Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111



Ciphertext	Plaintext
0000	1110
0001	0011
0010	0100
0011	1000
0100	0001
0101	1100
0110	1010
0111	1111
1000	0111
1001	1101
1010	1001
1011	0110
1100	1011
1101	0010
1110	0000
1111	0101

Block ciphers (cont.)

- Like a substitution on (very big) set of possible inputs
  - **If the block size is *n* bits, 2^n possible input values are mapped to 2^n output values**
    - like a *n*-bit substitution
  - **A way for encrypting could be to specify completely the mapping table (substitution table)**
    - permutation of 2^n *n*-bit inputs
    - there are $2^n!$ different possible transformations
  - **If n=64, would need table of 2^{64} entries storing 64-bit blocks**
    - 2^{64} 2^6 bit = 2^{70} bit = 2^{67}B = 2^{37}TB ≈ 10^{11}TB
    - it is too long
- Instead, a block cipher is created from smaller building blocks and a secret key
  - **if *n* is the cipher size and *m* is the key length, there are a total of 2^m possible transformations, rather than $2^n!$**

Block cipher (cont.)

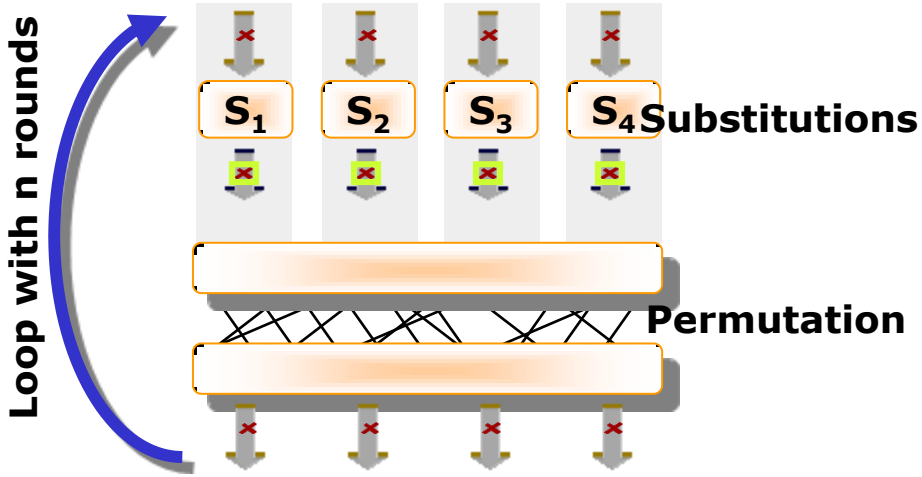
- How long should the plaintext block be?
  - **having block size too small**
    - in case of known-plaintext attack, an opponent may try to collect $\{M_i, C_i\}$ pairs and construct a decryption table
      - n-bit block cipher requires 2^n pairs
      - It is not required to find the key
    - In case of ciphertext–only attack, if a sequence of M_i have some properties (recognizable sequences of plaintext), it is possible to cryptanalyze the sequence of C_i
      - e.g. exploitation of language redundancy
  - **having block size too long, it could be inconvenient due to the increasing of complexity**
- 64-bit or 128-bit blocks are often used
  - **it is difficult to obtain all 2^{64} pairs (known-plaintext attack)**

Product ciphers

- Product cipher is a type of block cipher that works by executing in sequence a number of simple transformations such as substitution, permutation, and modular arithmetic
- Usually consist of iterations of several rounds of the same algorithm
  - **while the individual operations are not themselves secure, it is hoped that a sufficiently long chain would generate sufficient confusion and diffusion as to make it resistant to cryptanalysis**
- The operation must be reversible
- Important sub-classes of of product ciphers are:
  - **Feistel ciphers**
  - **SP-networks**

Substitution–permutation network

- SP-network is a product cipher that uses only substitutions and permutations
- One possible way to build a block cipher based on SP-network is
  - **break the input into managed-sized chunks (say 8 bits),**
  - **do a substitution on each small chunk,**
  - **and then take the output of all the substitutions and run them through a permuter (big as the input)**
  - **the process is repeated, so that each bit winds up as input to each substitution**
  - **each time is called *round***



Avalanche Effect

- Key desirable property of encryption algorithms
  - **where a change of one input or key bit results in changing approx half output bits**
    - Avalanche effect
- This is a characteristic of block ciphers
  - **block ciphers exhibit strong avalanche effect**
  - **stream cipher can just have a forward (avalanche) effect**
    - if XOR is used, just the corresponding ciphertext bit is affected

Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES)

- Block cipher designed to replace DES
  - **organized by National Institute of Standards and Technology (NIST)**
  - **NIST standard on November 26, 2001**
    - FIPS PUB 197 (FIPS 197)
  - **chosen from five candidate algorithms**
    - reviewed by US government (NSA), industry and academia
    - required a four-year process to pick the algorithm
    - winning algorithm chosen Oct 2, 2000
  - **also known as Rijndael block cipher**
    - original name of the algorithm submitted to AES selection process
    - developed by Joan Daemen and Vincent Rijmen (Belgium)
  - **currently, one of the most popular algorithms used in symmetric key cryptography**
    - also adopted as an encryption standard by the U.S. government

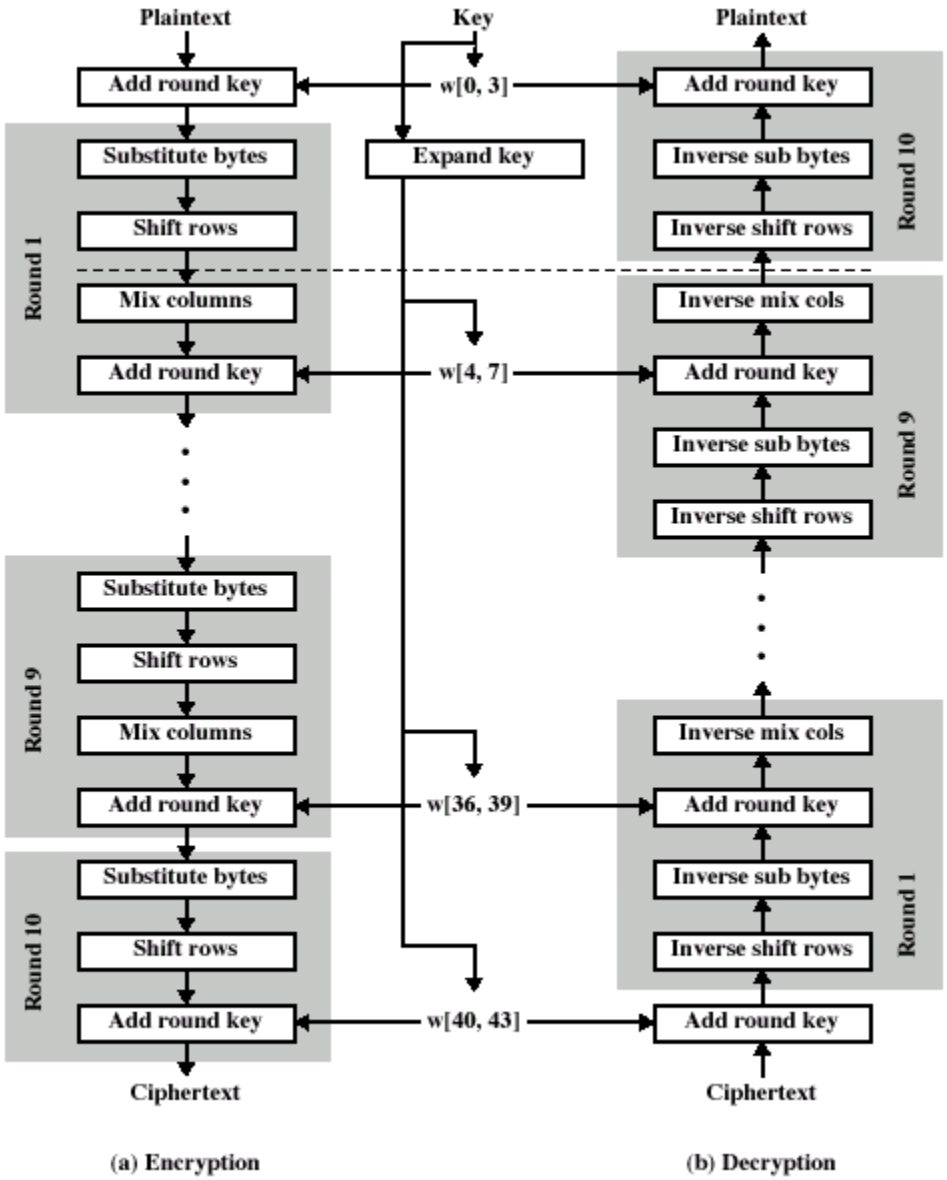
Advanced Encryption Standard (AES) (cont.)

- AES is not precisely the original Rijndael
  - **the original Rijndael algorithm supports a larger range of block and key sizes**
    - Rijndael can be specified with key and block sizes in any multiple of 32 bits, with a minimum of 128 bits and a maximum of 256 bits
- AES has fixed block size of 128 bits and a key size of 128, 192, or 256 bits
  - **i.e. block size: 16 bytes, key size: 16, 24, or 32 bytes**
- Rijndael is a substitution-permutation network
  - **not a Feistel network like DES**
- Fast in both software and hardware
  - **relatively easy to implement**
  - **requires little memory**

AES Description

- Due to the fixed block size of 128 bits, AES operates on a 4×4 array of bytes, termed the state
  - **versions of Rijndael with a larger block size had additional columns in the state**
- AES has 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys
- Most AES calculations are done in a special finite field

AES cipher

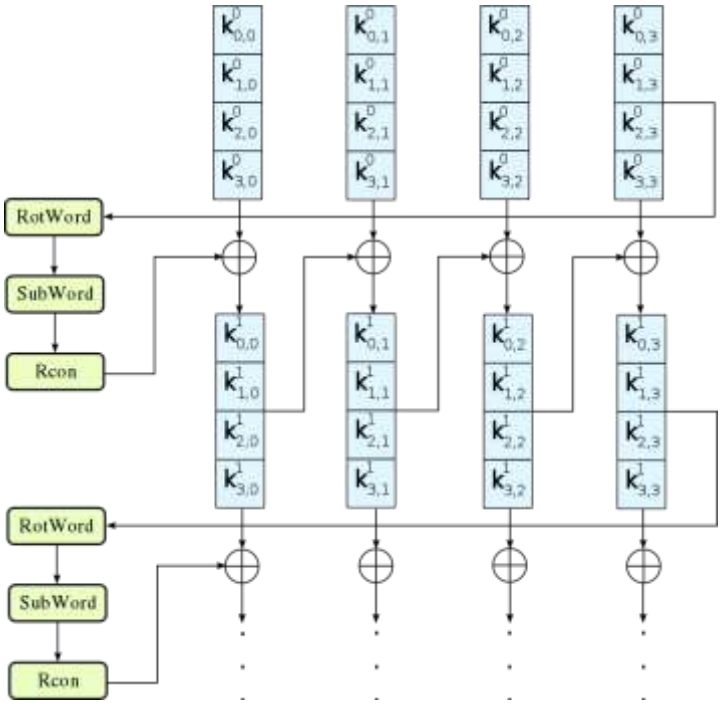

Plaintext
Add round key
Substitute bytes
Shift rows
Mix columns
Add round key
Round 1
Substitute bytes
Shift rows
Mix columns
Add round key
Round 9
Substitute bytes
Shift rows
Add round key
Round 10
Ciphertext
(a) Encryption
Key
w[0, 3]
Expand key
w[4, 7]
w[36, 39]
w[40, 43]
Plaintext
Add round key
Inverse sub bytes
Inverse shift rows
Round 10
Inverse mix cols
Add round key
Inverse sub bytes
Inverse shift rows
Round 9
Inverse mix cols
Add round key
Inverse sub bytes
Inverse shift rows
Round 1
Add round key
Ciphertext
(b) Decryption

AES cipher

- Algorithm
  - **KeyExpansion using Rijndael's key schedule**
  - **Initial Round**
    - AddRoundKey
  - **(*N*-1) Rounds (9 for 128bit key, 11 for 224bit key, 13 for 256bit key)**
    1. SubBytes — a non-linear substitution step where each byte is replaced with another according to a lookup table
    2. ShiftRows — a transposition step where each row of the state is shifted cyclically a certain number of steps
    3. MixColumns — a mixing operation which operates on the columns of the state, combining the four bytes in each column
    4. AddRoundKey — each byte of the state is combined with the round key derived from the cipher key using a key schedule
  - **Final Round (no MixColumns)**
    1. SubBytes
    2. ShiftRows
    3. AddRoundKey

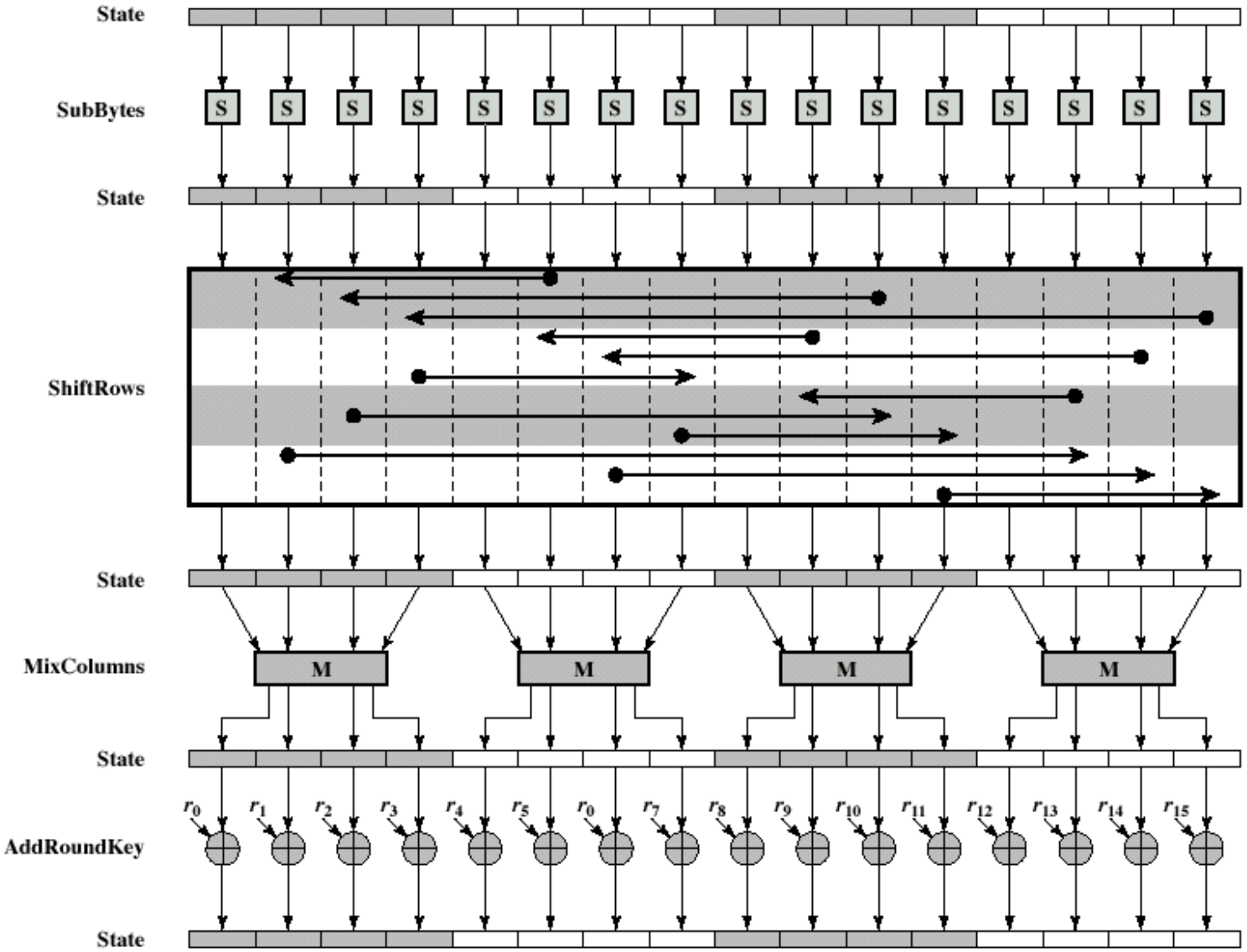
Key expansion

- Key Expansion routine to derive the (*r*+1) round keys from the cipher key
  - **where *r* is the number of rounds (9, 11, or 13)**
  - **RotWord**
    - one-byte left circular shift
  - **SubWord**
    - AES S-box substitution
  - **Rcon**
    - bitwise XOR with defined constant values



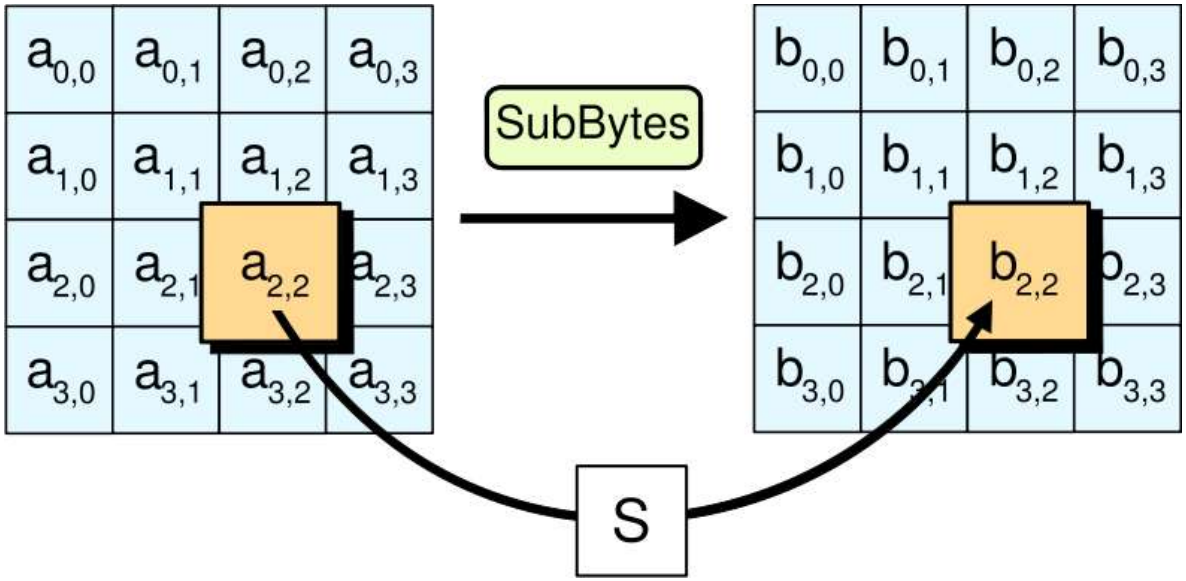
- Avalanche effect also on the key derivation
  - **one bit change of the encryption key should affect the round keys for several rounds**

AES single round



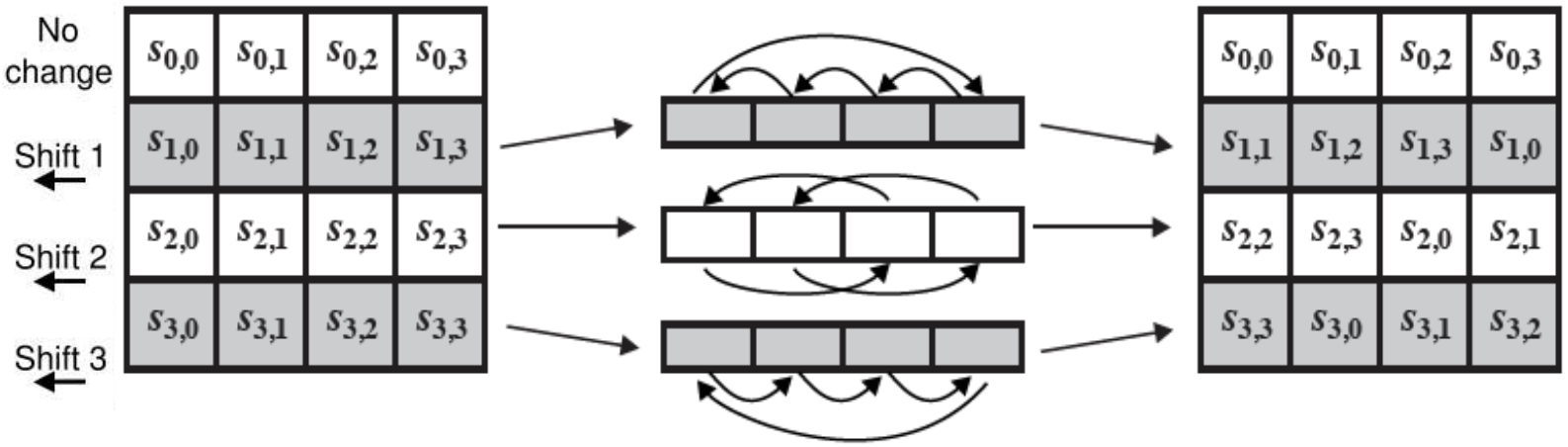
AES SubBytes step

- Each byte in the state is replaced using an 8-bit substitution box, the Rijndael S-box ($b_{ij} = S(a_{ij})$)
  - **S-box can be represented by a table of 256 8-bit values**
    - the transformation is a simple table lookup
  - **the S-box is derived from the multiplicative inverse over GF(2^8), known to have good non-linearity properties**
    - this operation provides the non-linearity in the cipher



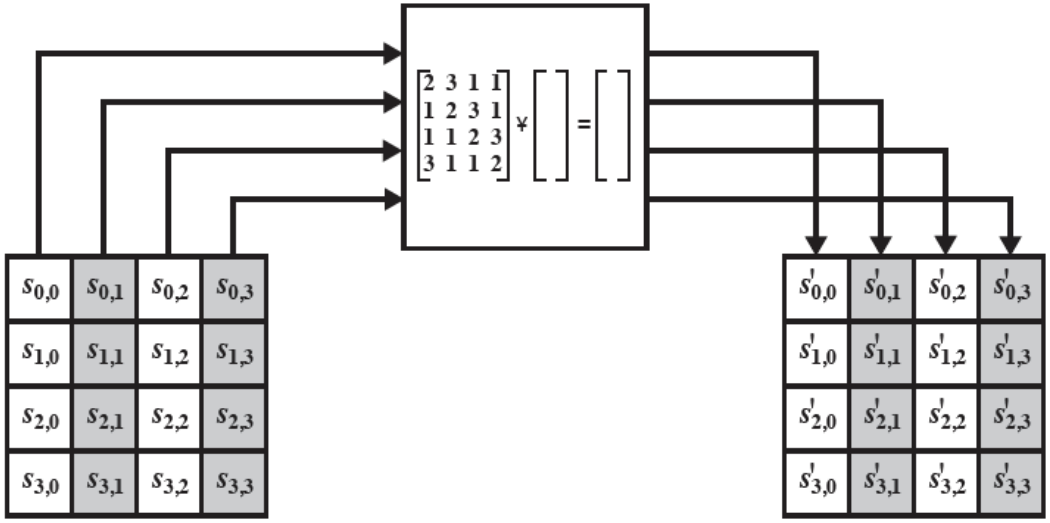
AES ShiftRows step

- ShiftRows step operates on the rows of the state
  - **it cyclically shifts the bytes in each row by a certain offset**
  - **for AES, the first row is left unchanged**
  - **each byte of the second row is shifted one to the left**
  - **similarly, the third and fourth rows are shifted by offsets of two and three respectively**



AES MixColumns step

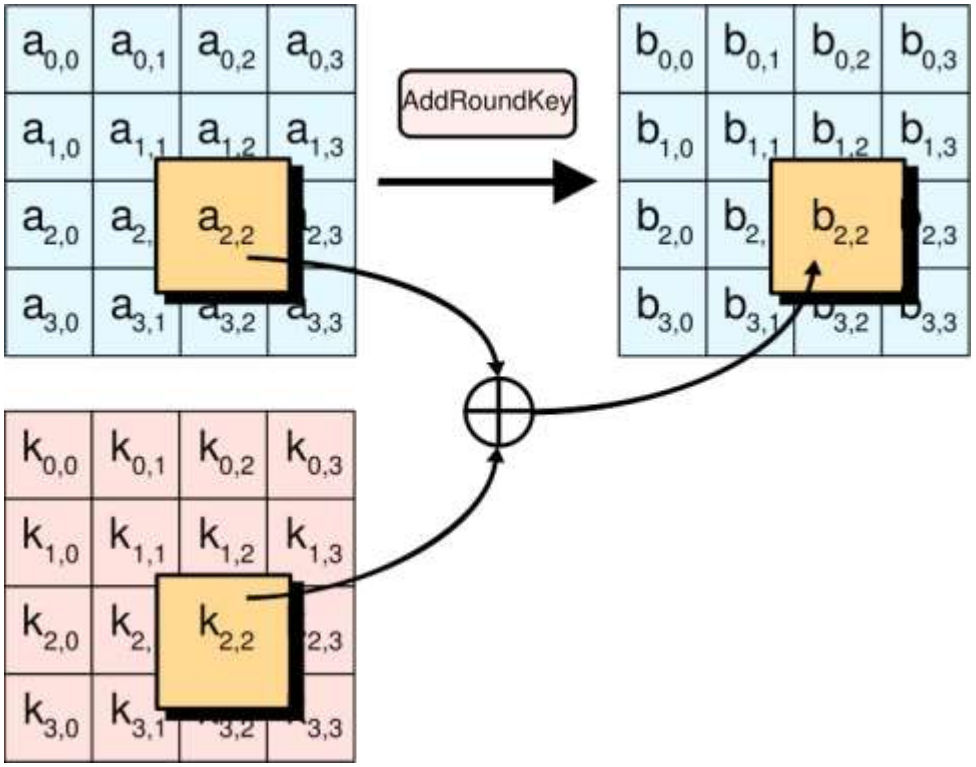
- Four bytes of each column of the state are combined using an invertible linear transformation
  - **each byte of column is a function of all four bytes in that column**
- Each column is treated as a polynomial and multiplied by a polynomial $c_i(x)=3x^3+x^2+x+2$ modulo x^4+1, with operations performed in a GF(2^8)
  - **byte addition is XOR, byte multiplication is polynomial multiplication reduced mod $x^8+x^4+x^3+x+1$**
- Can be viewed as a multiplication $S'=C \times S$ of a matrix C by the state S



- Together with ShiftRows, it provides diffusion in the cipher

AES AddRoundKey step

- The subkey is combined (XORed) with the state
  - **for each round, a subkey with same size as the state is derived from the main key (key schedule algorithm)**
  - **the subkey is added to the state using bitwise XOR**



AES Security

- In June 2003, the US Government announced that AES may be used also for classified information
  - **this marks the first time that the public has had access to a cipher approved by NSA for encryption of TOP SECRET information**
- The first key-recovery attacks on full AES were published in 2011
  - **the attack is faster than brute force by a factor of about four**
    - it requires $2^{126.1}$ operations to recover an AES-128 key
    - it requires $2^{189.7}$ operations to recover an AES-192 key
    - it requires $2^{254.4}$ operations to recover an AES-256 key
- The only successful attacks against AES have been side channel attacks
  - **however they require a lot of physical information from the system that executes the algorithm**

Multiple Encryption

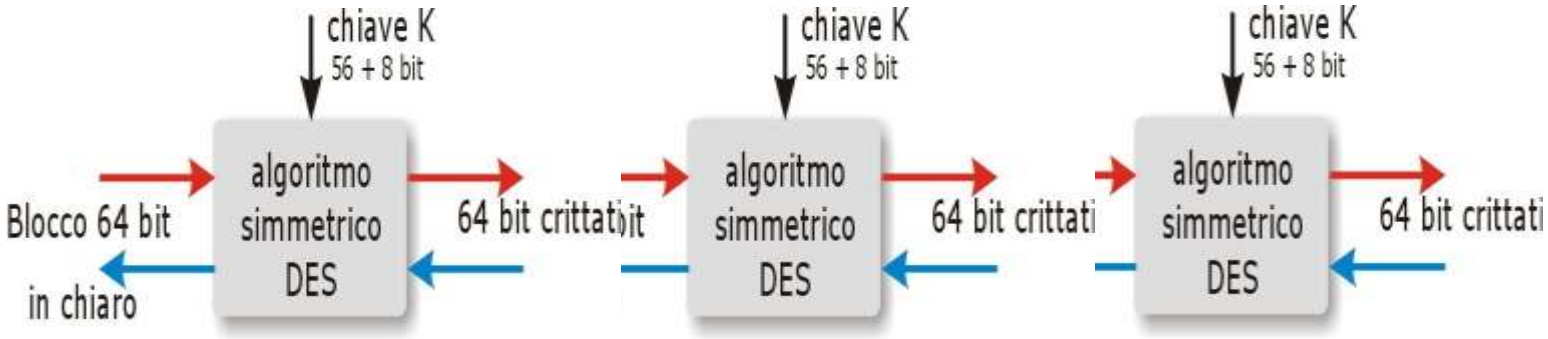
Multiple Encryption

- In general it is possible to increase the (computational) security of an encryption algorithm by using the same algorithm more times with different keys
  - **$c=E_{Kn}(\ldots(E_{K2}(E_{K1}(m)))$**
  - **$m=D_{K1}(D_{K2}(\ldots (D_{Kn}(c)))$**
- The used keys, together, can be seen as the overall secret key
  - **K=K1||K2||K3||…||Kn, with length = n |K1|**
- How many time should be performed? (2,3,4, ..)
  - **the more time the block is encrypted the more secure it is**
    - however for computation, no more encryptions than are necessary
- In most cases, both Encryption and Decryption algorithms can be see as encryption functions
  - **what combination of E and D can be chosen? (EEE, ED, etc)**

Multiple Encryption (cont.)

- Encrypting twice with the same key
  - **$c=E_K(E_K(m))$**
  - **no more secure that single encryption with K with length *n*: exhaustive search requires trying 2^n keys**
- Encrypting twice with two keys
  - **$c=E_{K2}(E_{K1}(m))$**
  - **there is an attack (not very practical), known as meet-in-the-middle, that breaks double encryption (EE) in roughly twice the time of a brute-force breaking single E**
    - since $X = E_{K1}(M) = D_{K2}(C)$
    - attack by encrypting *M* with all keys and store
    - then decrypt *C* with keys and match *X* value
    - can show takes $O(2^{n+1})$ encryptions and $O(2^n)$ memory
- Triple encryption with two/three keys (e.g. EEE, or EDE)
  - **meet-in-the-middle attack is not possible**

Example: Triple DES (3-DES)



- **Defined as alternative to the original DES with 56bit key**
- **$c = E_{K3}(D_{K2}(E_{K1}(m)))$**
- **Options:**
  - **3-DES with three different 56-bit keys: (k_1, k_2, k_3)**
    - key length: 56 + 56 + 56 = 168 bit (TDEA option 1)
  - **3-DES with two different 56-bit keys: (k_1, k_2, k_1)= (k_1, k_2)**
    - key length: 56+56 = 112 bit (TDEA option 2)
  - **3-DES with one 56-bit key: (k_1, k_1, k_1)= k_1**
    - equivalent to DES: $c = E_{K1}(D_{K1}(E_{K1}(m))) = E_{K1}(m)$

Stream ciphers based a block ciphers: Block cipher encryption modes

Encrypting large messages

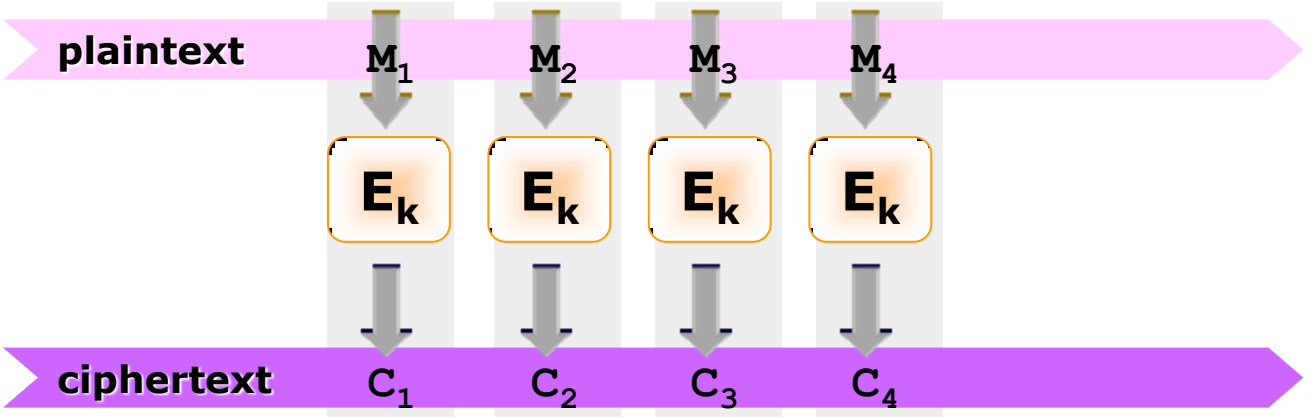
- Block ciphers encrypt fixed size blocks
  - **eg. AES has fixed block size of 128 bits**
- Usually we have arbitrary amount of information to encrypt (longer than the block size)
  - **need way to use in practice**
- Use of a block cipher for creating a stream cipher
  - **five modes have been defined for TDEA in: *NIST Special Publication 800-38A* (2001)**
    - Electronic Code Book (ECB)
    - Cipher Block Chaining (CBC)
    - Output Feedback (OFB)
    - Cipher Feedback (CFB)
    - Counter Mode (CTR)
  - **these schemes are applicable to any block cipher**
  - **other modes are also possible**

Padding

- If the encryption mode requires that the length of the whole message has to be a multiple of a given block size, a Padding operation has to be performed first
- Examples of padding algorithms:
  - **Bit padding (e.g. ISO/IEC 7816-4)**
    - symbol '1' (bit) is added, and then as many '0' bits as required are added
  - **ANSI X.923**
    - the last byte defines the number of padding bytes; the remaining bytes are filled with zeros
      - eg. [b1 b2 b3 00 00 00 00 05] (3 data bytes, then 5 bytes pad+count)
  - **RFC 5652 Cryptographic Message Syntax / PKCS#7 / PKCS#5**
    - the value of each added byte is the number of bytes that are added
      - eg. [b1 b2 b3 05 05 05 05 05]

Electronic Codebook (ECB) Mode

- Consist of doing the obvious thing, and it is usually the worst method
- The message is broken into n blocks of *b* with padding for the last one
  - **b = block size of the cipher**
  - **$n_* b$ = message size including padding**
- Each block is independently encrypted with the secret key K
  - **$C_i = E_K (M_i)$**
  - **$M_i = D_K (C_i)$**



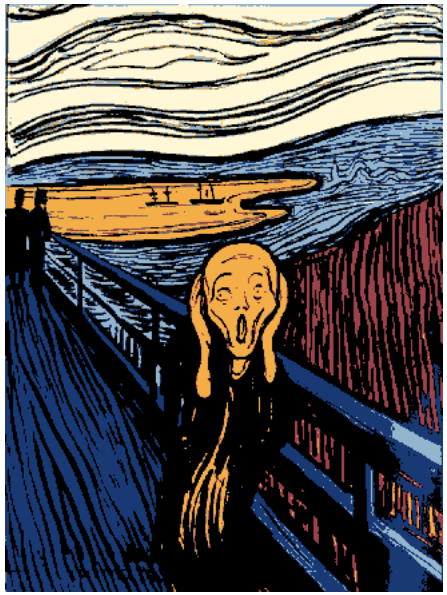
- Each block is a value which is substituted, like a codebook
- The cipher text has the same size of the plaintext (excluding padding)

Advantages and Limitations of ECB

- ECB is very simple and does not introduce extra operation (except for padding)
- There are a number of problems that arise and that don't show up in the single block case
  - **repetitions in message may show in ciphertext if aligned with message block**
    - if a message contains 2 identical blocks, the corresponding cipher blocks are identical; it can be a problem
      - in some cases it can be possible to guess a portion of the message
  - **by comparing two ciphertexts it is possible to discover similarities in the plaintexts**
    - no avalanche effect
  - **(partially) knowing the plaintext, is possible to rearrange the ciphertext blocks in order to obtain a new (known) plaintext**
- As result, ECB is rarely used
  - **main use is sending a few blocks of data (e.g. a secret key)**

Advantages and Limitations of ECB (cont.)

- Example of how repetitions may affect the security of ECB:



Original image
(3280x4400)



Image after ECB encryption

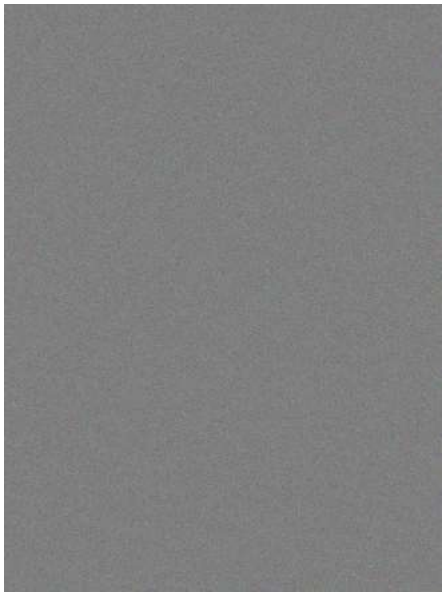
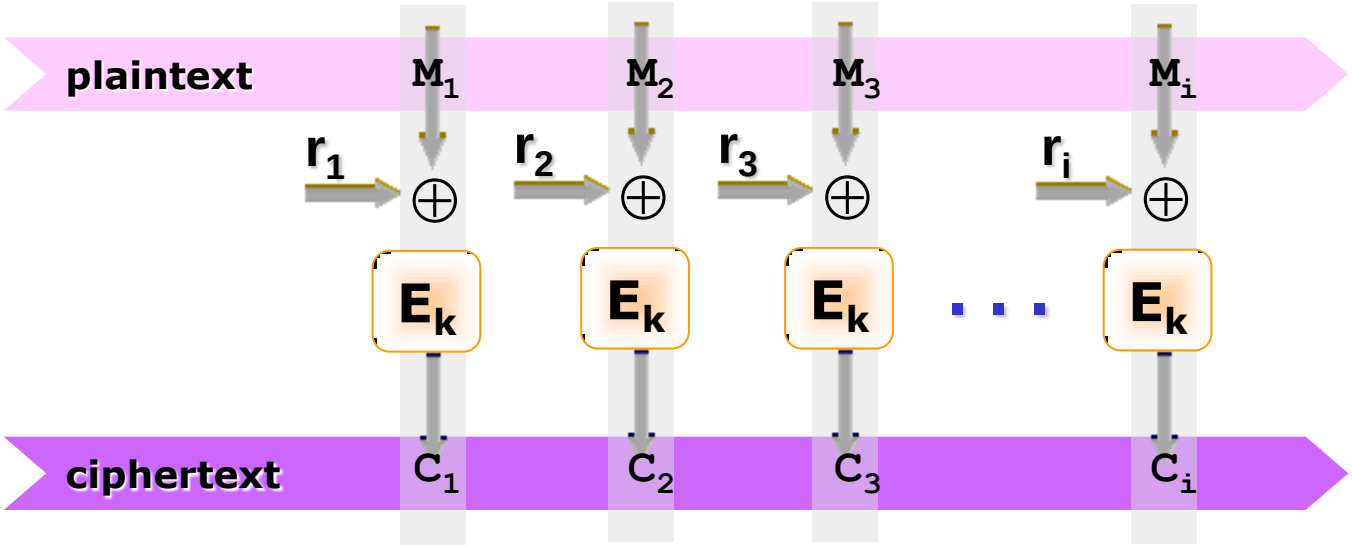


Image after CBC encryption

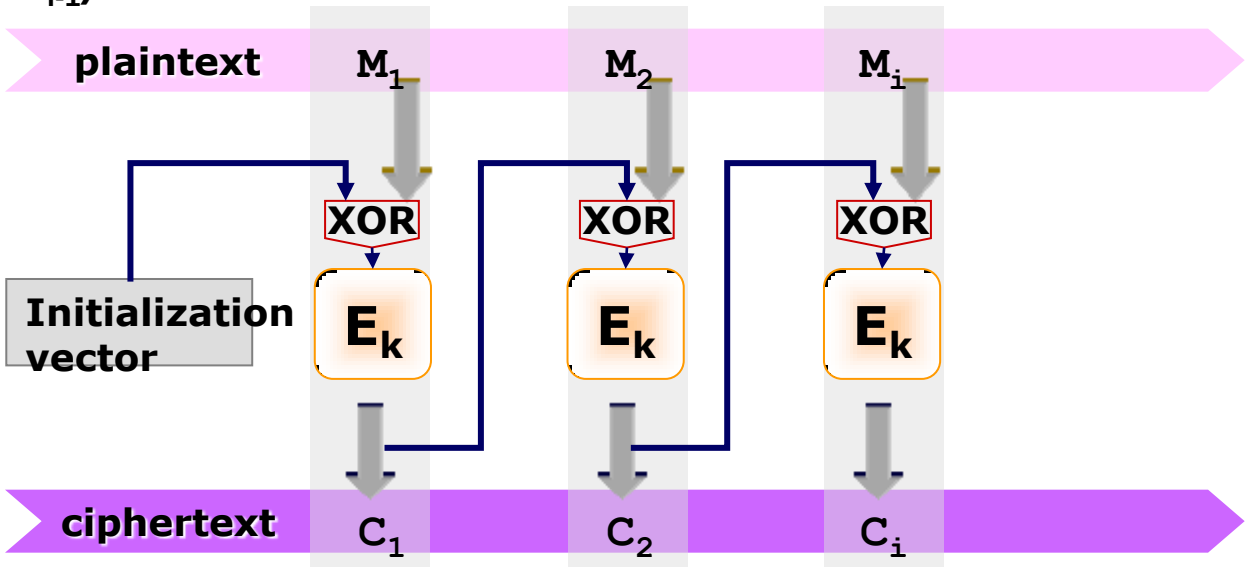
Cipher Block Chaining (CBC) Mode

- CBC objective: if the same block repeats in the plain text, it will not cause repeats of ciphertext
  - **avoiding some problems in ECB**
- How: adds a feedback mechanism to the cipher
- Result: plaintext is more difficult to manipulate
- Basic idea:



Cipher Block Chaining (CBC) Mode

- Plaintext patterns are concealed by XORing a block of *m* with the previous block of *c*
- Requires an IV (Initialization vector) of random data to encrypt the first block
  - $C_i = E_k(M_i \text{ XOR } C_{i-1})$
  - $C_0 = IV$



- Decryption is simple because ⊕ is reversible (A=B⊕C ⇔ A⊕C=B):
  - $M_i = D_k(C_i) \text{ XOR } C_{i-1}$

Advantages and Limitations of CBC

- CBC has the same performance of ECB, except for the cost of generating and transmitting the IV (and the cost of one ⊕)
- Each ciphertext block depends on **all** previous message blocks
  - **thus a change in the message affects all ciphertext blocks after the change as well as the original block**
    - forward avalanche effect
  - **a change in IV changes all bits of the ciphertext**

- As IV (e.g. 0), we could use a constant value however it may lead to some problems
  - **if two ciphertexts have the same initial blocks it means that the corresponding plaintext blocks are equal**
    - if a message is transmitted periodically, it is possible to guess whether changes have occurred, and in which block
  - **it is vulnerable to oracle attack**

Advantages and Limitations of CBC (cont.)

- The IV is not required to be a secret, and should not contain a secret
  - **it could be obtained by a chosen-ciphertext attack by asking to decrypt a ciphertext that starts with C1||C2 such that C1=C2, and take the resulting M1 and M2**
  - **then compute:**

    M1 xor M2 xor C1 =

    = (D_K(C1) xor IV) xor (D_K(C2) xor C1) xor C1 =

    = D_K(C1) xor D_K(C1) xor IV =

    = IV

CBC Threats: Modifing ciphertext blocks

- Using CBC correctly does not eliminate the problem of someone modifying the message in transit
- If IV is sent in the clear, an attacker can change bits of the first plaintext block (working on the ciphertext and IV) by simply changing the IV
  - **changing bit *h* of IV has predictable effect to bit *h* of M_1**
  - **hence either IV must be a fixed value or it must be sent encrypted in ECB mode before rest of message**
    - e.g. by changing IV $\rightarrow$ IV' $\equiv$ IV$\oplus$X, when decrypting we have:

      $M_1' = D_K(C_1) \oplus IV' = (M_1 \oplus IV) \oplus IV' = M_1 \oplus IV \oplus IV \oplus X = M_1 \oplus X$
- If the attacker changes the ciphertext block C_i, C_i gets $\oplus$'d with the decrypted C_{i+1} to yield M_{i+1}
  - **since M_{i+1} is obtained as: $M_{i+1} = D_K(C_{i+1}) \oplus C_i$**
  - **changing bit *h* of C_i has predictable effect to bit *h* of decrypted M_{i+1}**
  - **however the attacker cannot know the new decrypted M_i' (a new random block, as side effect)**
    - e.g. by changing $C_i \rightarrow C_i' \equiv C_i \oplus X$, when decrypting we have:

      $M_i' = D_K(C_i') \oplus C_{i-1} = D_K(C_i \oplus X) \oplus C_{i-1}$ = unpredictable without knowing *K*

      $M_{i+1}' = D_K(C_{i+1}) \oplus C_i' = (M_{i+1} \oplus C_i) \oplus C_i' = M_i \oplus X$

CBC Threats: Rearranging ciphertext blocks

- Knowing the plain text, the corresponding ciphertext and IV, it is possible to rearrange the C_1, C_2, C_3, .. (building blocks), in such a way to obtain a new known M'_1, M'_2, M'_3 ..
  - **e.g. suppose an intruder rearranges the ciphertext $c=C_1,C_2,C_3,C_4$ to $c'=C_1,C_3,C_3,C_4$, then when decrypting:**

$$M_1' = D_K(C_1') \oplus IV' = D_K(C_1) \oplus IV = M_1$$
$$M_2' = D_K(C_2') \oplus C_1' = D_K(C_3) \oplus C_1 = (M_3 \oplus C_2) \oplus C_1$$
$$M_3' = D_K(C_3') \oplus C_2' = D_K(C_3) \oplus C_3 = (M_3 \oplus C_2) \oplus C_3$$
$$M_4' = D_K(C_4') \oplus C_3' = D_K(C_4) \oplus C_3 = M_4$$

  - **other example: if the modified ciphertext is $c'=C_1,C_3,C_2,C_4$, then:**

$$M_1' = D_K(C_1') \oplus IV' = D_K(C_1) \oplus IV = M_1$$
$$M_2' = D_K(C_2') \oplus C_1' = D_K(C_3) \oplus C_1 = (M_3 \oplus C_2) \oplus C_1$$
$$M_3' = D_K(C_3') \oplus C_2' = D_K(C_2) \oplus C_3 = (M_2 \oplus C_1) \oplus C_3$$
$$M_4' = D_K(C_4') \oplus C_3' = D_K(C_4) \oplus C_2 = (M_4 \oplus C_3) \oplus C_2$$

- These threats can be combated by adding a MIC (message integrity check) code, or a strong checksum to the plaintext before encrypt
  - **use of 16 bit checksum doesn't solve the problem, since 1 in 2^{16} chance that the checksum will work**
    - after 2^{15} attempts, the probably of obtaining a correct checksum is ~50%

Output Feedback (OFB) Mode

- Starting from a IV, a sequence of O_i block is obtained as

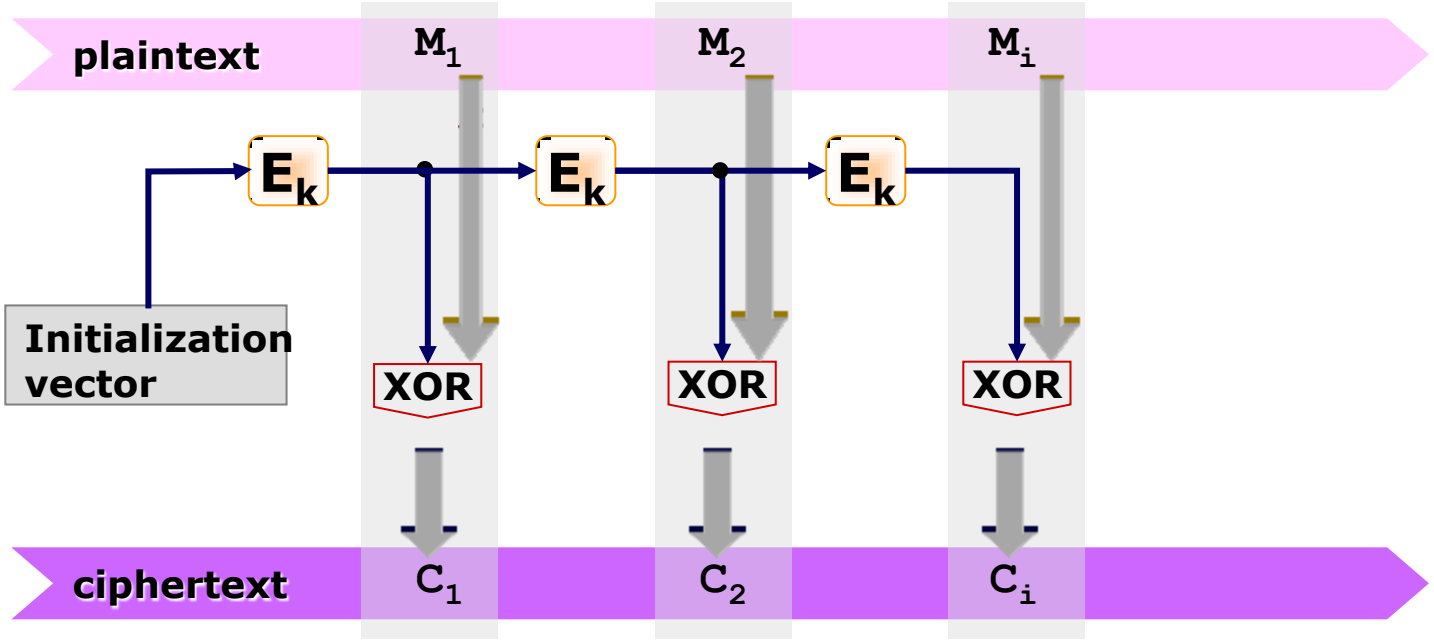
$$\begin{cases} O_0 = IV \\ O_i = E_K(O_{i-1}) \end{cases}$$

  - **each block in encrypted by**

    $C_i = M_i \text{ XOR } O_i$

  - **decryption**

    $M_i = C_i \text{ XOR } O_i$



Output Feedback (OFB) Mode

- How it works:
  - **A pseudorandom number O_0 is generated (named IV as in CBC)**
  - **O_0 is encrypted (using secret key K) obtaining O_1**
  - **from O_1 is obtained O_2 and so on, as many block are needed**

    $O_i = E_K(O_{i-1})$
    $O_0 = IV$
  - **plaintext in encrypted by by ⊕ing it with the pseudorandom stream (one-time pad) generated by the OFB**
  - **the same for the ciphertext**
- Given the pseudorandom keystream $o = O_1\ O_2\ O_3\ \ldots\ O_n$

  c = m XOR o

  m = c XOR o
- Message is treated as a stream of bits
  - **OTP acts as a stream cipher based on XOR**
- Example of uses: stream encryption over noisy channels

Advantages and Limitations of OFB

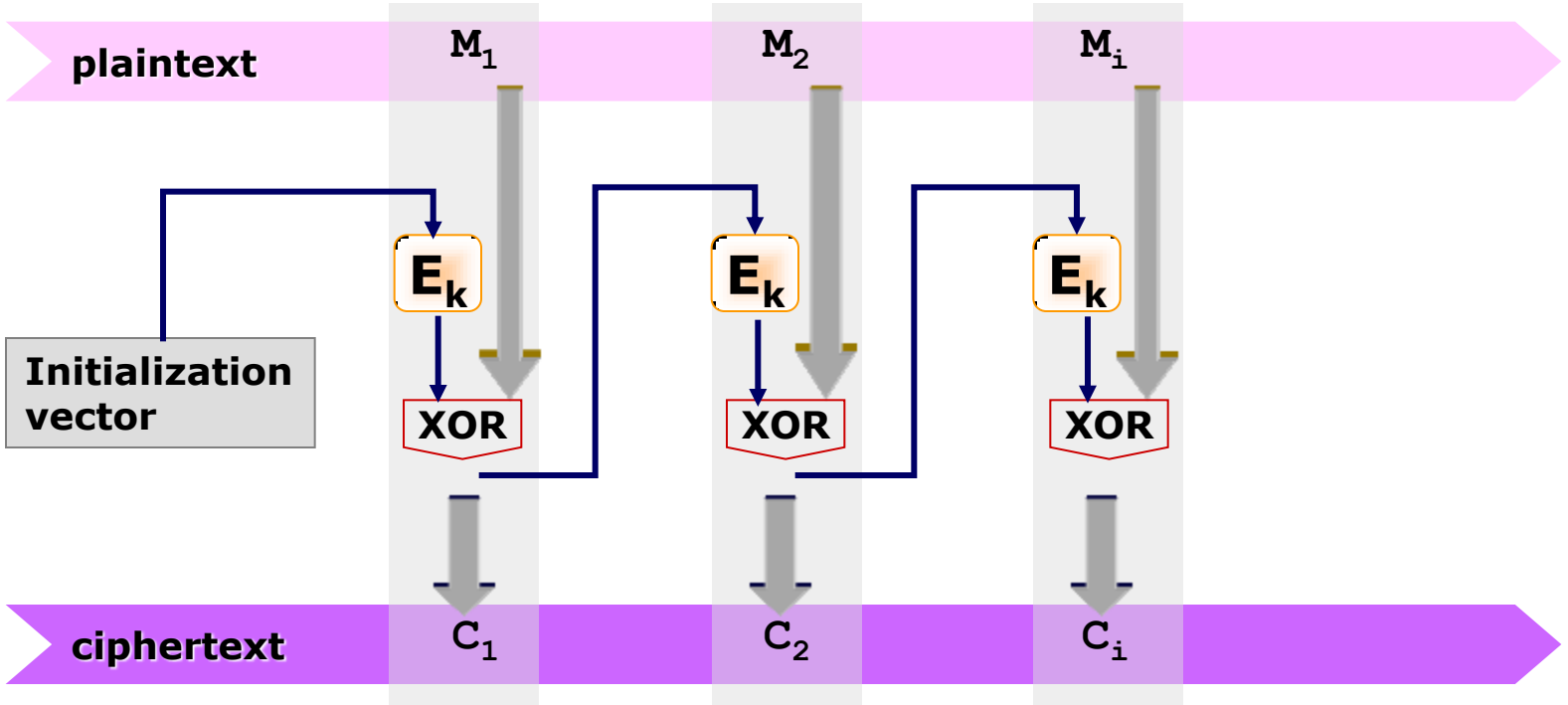
- Advantages of OFB:
  - **keystream can be generated in advances**
  - **if a message arrives in arbitrary-sized chucks, the associated ciphertext can immediately be transmitted**
  - **possibility to encrypt variable-length messages: no need of padding**
  - **if some bits of the ciphertext get garbled, only those bits of plaintext get garbled (no error propagation)**
- Disadvantages of OFB:
  - **if the plaintext is known by a bad guy, he can modify the ciphertext forcing the plaintext into anything he wants**
  - **if the plaintext is known, the keystream can be easily obtained**
    - hence must never reuse the same sequence (key+IV)

Cipher Feedback (CFB) Mode

- Similar to OFB
- The *b* bits shifted in to the encryption module are the *b* bits of the ciphertext from the previous block

$$\begin{cases} C_0 = IV \\ C_i = M_i \text{ XOR } E_K(C_{i-1}) \end{cases}$$

$$M_i = C_i \text{ XOR } E_K(C_{i-1})$$



Advantages and Limitations of CFB

- The block cipher is used in encryption mode at both ends
  - **like in OFB**
  - **however:**
    - the one-time pad cannot be generated entirely in advance
      - needs to stall while do block encryption after every *b* bits
- Errors in cipher text propagate to the next block
  - **like in CBC**
- The lost of a portion of the ciphertext can be resumed if it is multiple of s-bit used by the CFB
  - **it is possible to have *s*-bit CFB with *s* different from *b* (the $E_k(\cdot)$ size), e.g. 8 bits**
    - with OFB or CBC if octects (bytes) are lost in transmission or extra octects are added, the rest of transmission is garbled
      - sender and receiver must remain in sync or recovery method is needed
    - with 8-bit CFB as long as an error is an integral number of octects, things will be resynchronized

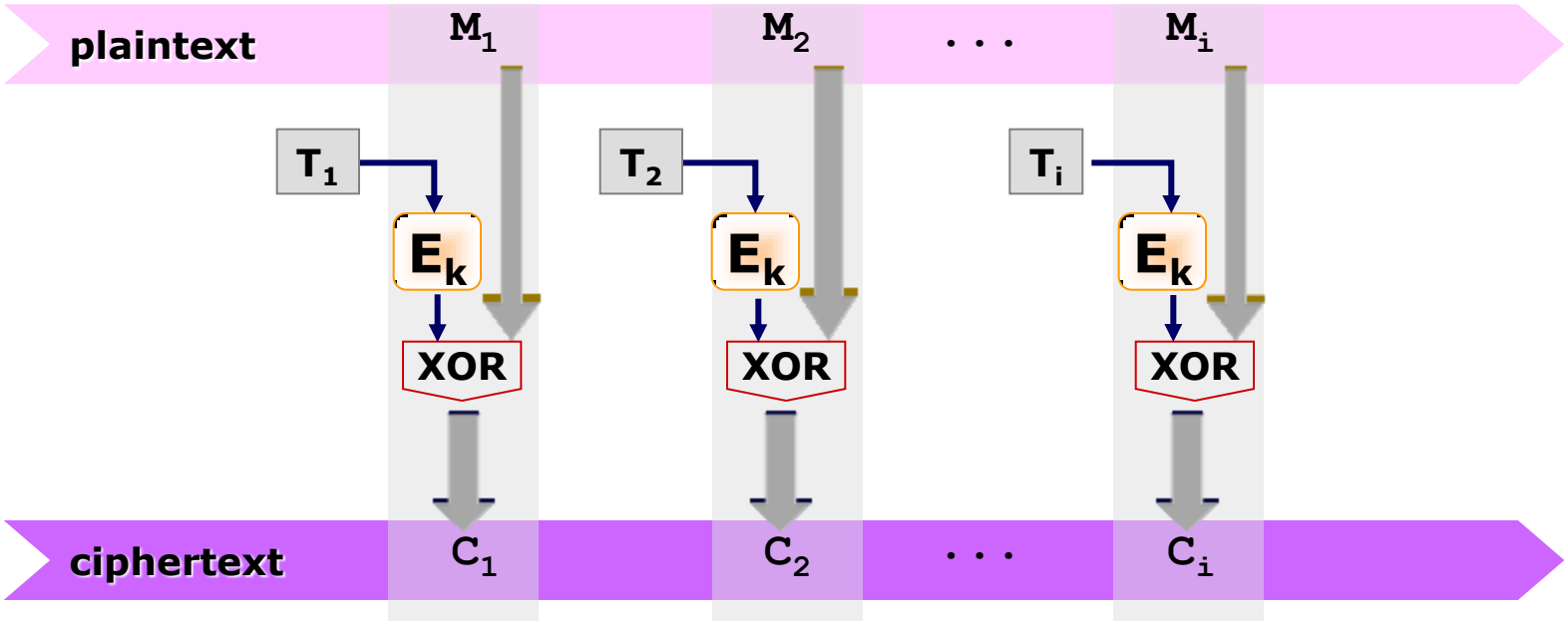
Counter (CTR) Mode

- Similar to the OFB
- The CTR output blocks are generated by encrypting a set of input blocks T_i called counters

  $O_i = E_K(T_i)$

  $C_i = M_i \text{ XOR } O_i$

  - **the sequence of counters $T_1, T_2, .., T_i$ must have the property that each block in the sequence is different from every other block**



Counter (CTR) Mode (cont.)

- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block (modulo 2^n, where n is the block size)

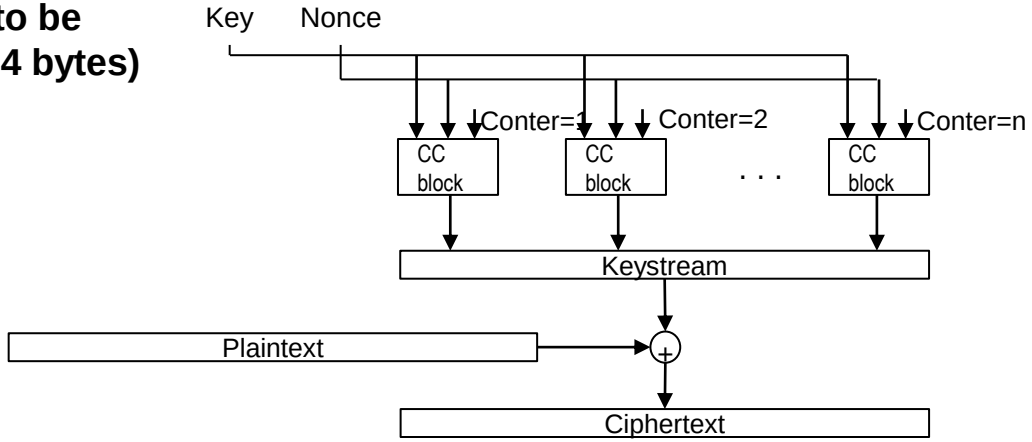
- Example with 16-byte block cipher:
  - **take a random number, for example "12345678"**
  - **T1 = 12345678**00000000
  - **T2 = 12345678**00000001
  - **...**
  - **Ti = 12345678**0000000 i
  - **go on until you reach the desired number of blocks**

Counter (CTR) Mode (cont.)

- Advantages:
  - **the forward cipher functions can be applied to the counters prior to the availability of the plaintext or ciphertext data**
    - like OFB
  - **the cipher functions can be performed in parallel**
  - **the plaintext block that corresponds to any particular ciphertext block can be recovered independently from the other plaintext blocks if the corresponding counter block can be determined**

Stream ciphers: ChaCha20

- Stream cipher proposed by Daniel J. Bernstein (in 2008)
  - **refinement of Salsa20 stream cipher (2005, same author)**
- Encryption/decryption performed by XOR of a keystream with the plaintext/ciphertext
  - **uses a 256-bit key, 32-bit initial counter, 96-bit nonce (IV)**
  - **keystream is the concatenation of 512-bit keystream blocks generated by a block function**
  - **ChaCha20 block function takes as input the key, current counter, and nonce values, and gives as output a 512-bit keystream block**
    - internally works in rounds (20) on a internal state
    - the initial state is formed by the concatenation of 128-bit constant, 256-bit key, 32-bit counter, and 96-bit nonce
  - **no requirement for the plaintext to be an integral multiple of 512 bits (64 bytes)**



Oracle Attacks

Oracle Attack

- An oracle attack involves an attacker interacting with a system/service (the oracle) that can provide information without directly solving the problem
  - **may encrypt or decrypt arbitrary messages without revealing the key, or provide other information (e.g. whether an operation succeeded or not)**

Oracle Attack (cont.)

- Some elements are required to exploit an oracle:
  - **ability to submit messages (plaintext or ciphertext)**
  - **ability to observe the response**
  - **ability to repeat these actions**

Example of Encrypting Oracle Attack

- Suppose that you have a ciphertext you want to decrypt (without knowing the key *K* of length *n*) and you know a portion of the corresponding plaintext, and only *h* bits are unknown
  - **a brute force attack would still require 2^n attempts (*n=len(K)*), infeasible**
- If you have access to an oracle that encrypts input messages using the same key, and if the encryption algorithm *c=E(K,m)* is deterministic
  - **a chosen-plaintext attack (CPA) may be possible, requiring 2^h attempts**

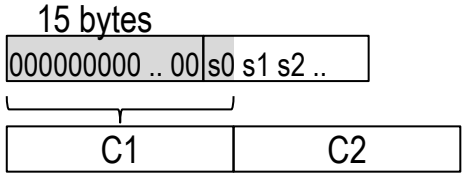
Example of ECB Oracle Attack

- Example: CPA against a ECB encryption oracle encrypting a message concatenated to a secret
  - **suppose that the encryption oracle gives the following output: E_K(m||secret)**
    - E_K() is a block cipher in ECB mode, e.g. block size 16
    - *m* is the input message sent to the oracle, *secret* is the message that you want to discover
  - **strategy of the attack:**
    - send an input message *m* that is 1 byte shorter than the block size and save the result
    - bruteforce the last byte until you find the same ciphertext block
    - repeat the previous steps bruteforcing the entire secret, one byte at time

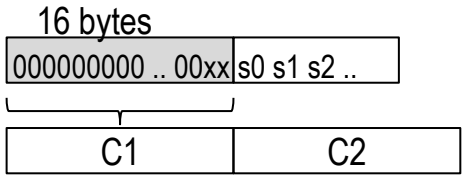
Example of ECB Oracle Attack (cont.)

- Attack:
  - **1) send 15 bytes (m), e.g. b0 b1 b2 .. b14**
    - the first block of the response of the oracle will be:

      $C1 = E_K('b0\ b1\ b2\ ..\ b14 \| s0')$

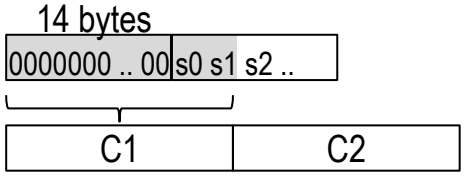


  - **2) try to send the following inputs (16 bytes):**
    - b0 b1 b2 .. b14 0x00
    - b0 b1 b2 .. b14 0x01
    - …
    - until the response is *C1*; the last byte of this input message is s0

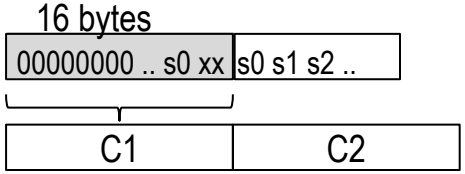


  - **3) send 14 bytes (m), b0 b1 b2 .. b13**
    - the first block of the response of the oracle will be:

      $C1 = E_K('b0\ b1\ b2\ ..\ b13 \| s0\ s1')$



  - **4) since you already know s0, search s1 by sending the following inputs:**
    - b0 b1 b2 .. b13 s0 0x00
    - b0 b1 b2 .. b13 s0 0x01
    - …
    - until the response is *C1*; the last byte of this input message is s1



  - **5) go on byte after byte until all bytes of the secret are discovered**

Example of ECB Oracle Attack (cont.)

- If the secret is longer that 16 bytes, after discovered the first 16 bytes you have to move to the second block, third block, etc.

```
0000000000000000? ???????????????? ???????????????? ????????????????...
00000000000000A? ???????????????? ???????????????? ????????????????...
0000000000000AB? ???????????????? ???????????????? ????????????????...
000000000000ABC? ???????????????? ???????????????? ????????????????...
00000000000ABCD? ???????????????? ???????????????? ????????????????...
0000000000ABCDE? ???????????????? ???????????????? ????????????????...
000000000ABCDEF? ???????????????? ???????????????? ????????????????...
00000000ABCDEFG? ???????????????? ???????????????? ????????????????...
0000000ABCDEFGH? ???????????????? ???????????????? ????????????????...
000000ABCDEFGHI? ???????????????? ???????????????? ????????????????...
00000ABCDEFGHIJ? ???????????????? ???????????????? ????????????????...
0000ABCDEFGHIJK? ???????????????? ???????????????? ????????????????...
000ABCDEFGHIJKL? ???????????????? ???????????????? ????????????????...
00ABCDEFGHIJKLM? ???????????????? ???????????????? ????????????????...
0ABCDEFGHIJKLMN? ???????????????? ???????????????? ????????????????...
ABCDEFGHIJKLMNO? ???????????????? ???????????????? ????????????????...
00000000000000A BCDEFGHIJKLMNOP? ???????????????? ????????????????...
00000000000000AB CDEFGHIJKLMNOPQ? ???????????????? ????????????????...
0000000000000ABC DEFGHIJKLMNOPQR? ???????????????? ????????????????...
000000000000ABCD EFGHIJKLMNOPQRS? ???????????????? ????????????????...
```

Example of ECB Oracle Attack (cont.)

Notes:

- This attack works also with CBC, in case fixed IV is used

- In case of ECB, the step (2), can be done with a single 256-block chosen plaintext query to the encryption oracle
  - **all 256 possible blocks can be sent concatenated in one message (256 x 16 = 4096 bytes)**
  - **similar also for step 4 and following**

Padding Oracle

- In some cases an attacker can modify a ciphertext and checking whether an unpadding error occurs when decrypting the ciphertext
  - **in general it is always possible to decrypt a tampered ciphertext, however an error may occur when unpadding**
    - due to malformed padding value
- We call Padding Oracle a server that receives a ciphertext and tells whether the padding is correct

CBC Threats: Padding Oracle Attack

- The attack targets the padding check done during decryption
  - **the attacker can exploit error messages or differences in response time when an incorrect padding is detected**
- Outline:
  - **the attacker has access to ciphertext (but not the key)**
  - **the attacker sends modified ciphertext to the server and observes whether the server accepts or rejects the decrypted message based on the padding**
    - valid padding: the decryption will succeed, and the server will respond accordingly
    - invalid padding: the decryption fails, and the server will respond with an error, indicating that the padding is incorrect
  - **the attacker exploits this behavior by trying to guess the padding byte by byte**
    - by modifying the ciphertext and observing the responses, the attacker can deduce the correct padding and then work backwards to reveal the plaintext

CBC Threats: Padding Oracle Attack (cont.)

- Example:
  - **consider the following ciphertext for two blocks:**
    - C1 = ciphertext block 1
    - C2 = ciphertext block 2, the target
  - **the attacker will focus on the second block (C2) and try to modify the first block (C1) such that when decrypted, the padding of the second block becomes valid**
  - **if the padding scheme is PKCS#7, it uses known values (like 0x01, 0x02, etc.), then the attacker can craft different guesses for the padding by modifying C1**
  - **through this process, the attacker can recover the original plaintext, block by block, without needing the encryption key**

CBC Threats: Padding Oracle Attack (cont.)

- Steps:
  - **1) select a random C1 and try different values for the last byte C1[15] (supposing block length=16 bytes), and stop when a valid pad is obtained (the decryption/unpadding succeeded)**
    - let's suppose that the valid pad is one byte 0x01 (see the note below), then it means that:
    - $C1[15] \oplus M2[15] = 0x01 \Rightarrow M2[15] = C1[15] \oplus 0x01$
  - **2) now set C1[15]= M2[15] ⊕ 0x02, try different values for C1[14] and stop when a valid pad is obtained**
    - $C1[14] \oplus M2[14] = 0x02 \Rightarrow M2[14] = C1[14] \oplus 0x02$
  - **3) repeat with C1[16-i], M2[16-i], pad [i]*i, until all M2 bytes are discovered**
- Note: at step 1 a success may occur also in case the valid pad is 2 bytes 0x0202, or 3 bytes 0x030303, or longer
  - **for discover which pad occurred, try to change C1[0], then C1[1], and so on until a wrong pad is obtained when chaning C1[i], then:**
    - len(pad) = n = (16-i), and pad that is n||n||n,..,||n
    - M2[i:16]= C1[i:16] ⊕ [n]*n
  - **the remaining bytes of M2 can be obtained following step 3**

CBC Threats: Padding Oracle Attack (cont.)

- Mitigations:
  - **do not leak whether a padding error occurred**
    - e.g. constant-time error responses
      - always respond with the same time for valid and invalid padding
  - **authenticated encryption**
    - using an authenticated encryption mode prevents the attacker from distinguishing between valid and invalid ciphertexts
      - in addition, both confidentiality and integrity are provided

- This attack is possible when CBC mode is improperly implemented or when it's not used in conjunction with the right mitigations